Small Radio Telescope Drive Controller

Technical Documentation and Operations Manual

Version 2.0

Acre Road Observatory
University of Glasgow

Built 2026-08-19

Describes commit `cd787d7` of 2026-08-19

Abstract

This document is the definitive technical description of the Small Radio Telescope (SRT) drive and control system used for 21 cm hydrogen line observations at Acre Road Observatory, Glasgow. The system comprises an Arduino Due carrying out real-time motor control, a WT32-ETH01 (ESP32) controller performing astronomical coordinate transformation, pointing correction and network service, a GNU Radio receiver, and a Flask observation scheduler that also runs the pointing calibration. The document covers the hardware, the firmware on both microcontrollers, the coordinate and pointing mathematics in full, the resident pointing model and the Sun-scan calibration that determines it, the private Ethernet link between the observatory computer and the controller, the receiver and scheduler, and the installation, operation and fault-finding procedures.

Contents

1 Introduction **6**
1.1 Purpose and scope . . . 6
1.2 System overview . . . 6
1.3 Two coordinate frames . . . 6
1.4 Document organisation . . . 7

2 System architecture **7**
2.1 Architectural overview . . . 7
2.2 Data flow . . . 8
2.3 Communication interfaces . . . 8

3 Hardware configuration **8**
3.1 Arduino Due microcontroller . . . 8
3.1.1 Pin assignments . . . 9
3.2 Motor drive system . . . 9
3.2.1 H-bridge motor drivers . . . 9
3.2.2 Position encoders . . . 10
3.2.3 Current sensing . . . 10
3.3 ESP32 controller . . . 10
3.3.1 WT32-ETH01 to Arduino Due wiring . . . 10
3.4 Operational limits . . . 11

4 Arduino Due firmware **11**
4.1 Firmware overview . . . 11
4.2 State machine . . . 12
4.3 Homing sequence . . . 13
4.4 Motion profiles . . . 13
4.4.1 Acceleration ramp . . . 13
4.4.2 Deceleration ramp . . . 13
4.5 Stall and limit handling . . . 14
4.6 Serial command interface . . . 14
4.6.1 Configurable parameters . . . 15
4.7 Status output format . . . 15
4.8 Fault detection . . . 16
4.9 Simulation mode . . . 16

5 ESP32 controller **16**
5.1 Software architecture . . . 16
5.2 Settings . . . 17
5.3 Networking . . . 18
5.3.1 Network modes . . . 18
5.3.2 Clock discipline . . . 18
5.3.3 DNS resilience . . . 19
5.4 HTTP API . . . 19
5.5 Stellarium protocol . . . 19
5.6 Tracking . . . 20
5.6.1 Below-horizon handling . . . 20
5.6.2 Azimuth wrap-around and the upper stop . . . 20
5.6.3 Galactic-plane target . . . 20

6 Coordinate transformations **21**
6.1 Reference frame . . . 21
6.2 Conversion pipeline . . . 21
6.2.1 Julian date . . . 21
6.2.2 Sidereal time . . . 21
6.3 Precession . . . 21
6.4 Horizontal coordinates . . . 22
6.5 Galactic coordinates . . . 22
6.6 Solar position . . . 22
6.7 Lunar position . . . 22
6.8 Accuracy of the transforms . . . 22

7 Pointing model and calibration **23**
7.1 Atmospheric refraction . . . 23
7.2 The model . . . 23
7.3 The forward and inverse transforms . . . 24
7.4 Guards . . . 24
7.5 Where the calibration lives . . . 24
7.6 Measuring the model: the Sun scan . . . 25
7.6.1 Raster geometry . . . 25
7.6.2 Beam fit . . . 25
7.6.3 The datum: an absolute (T, D) pair . . . 25
7.7 Fitting the model . . . 25
7.7.1 Design matrix . . . 25

7.7.2 Weighted solution . . . 26
7.7.3 Geometry and quality gates . . . 26
7.8 Wire format and application . . . 27
7.9 Operating procedure: a calibration day . . . 27

8 Observatory host and the controller link **28**
8.1 What the link is . . . 28
8.2 Why . . . 28
8.3 Address plan . . . 28
8.4 Hardware . . . 28
8.5 Building the link . . . 29
8.6 Firewall . . . 29
8.7 Verification . . . 30
8.8 Recovery . . . 30
8.9 What the link deliberately does not do . . . 31

9 Beam, radiometry and velocity frames **31**
9.1 Beam and aperture . . . 31
9.2 System temperature . . . 32
9.3 The radiometer equation . . . 32
9.4 Temperature calibration . . . 32
9.4.1 Single-point blank-sky calibration . . . 32
9.4.2 Noise diode and the Y-factor . . . 33
9.5 Velocity frames — and the LSR correction the system does not apply . . . 33

10 H1 receiver and observation scheduler **34**
10.1 Receiver overview . . . 34
10.2 HDF5 output format . . . 34
10.3 Observation scheduler . . . 35
10.3.1 Interface . . . 35
10.3.2 Runtime behaviour . . . 35
10.3.3 Coordinate systems and drift scans . . . 36
10.3.4 Observation parameters . . . 37
10.3.5 Automated workflow . . . 37
10.3.6 Tests . . . 37

11 Sky simulator **38**
11.1 Sky models . . . 38
11.2 Instrument model . . . 38
11.3 Velocity frames . . . 38
11.4 Modes . . . 39
11.5 Realise . . . 39

12 Installation **39**
12.1 Prerequisites . . . 39
12.2 Arduino Due drive firmware . . . 39
12.3 WT32-ETH01 controller firmware . . . 40
12.4 Configuration . . . 40
12.5 Receiver prerequisites . . . 40

13 Testing and verification **40**
13.1 Drive firmware in simulation . . . 41
13.2 Native unit tests . . . 41
13.3 Scheduler and calibration tests . . . 41
13.4 The Due emulator . . . 41

14 Operation **42**
14.1 Startup sequence . . . 42
14.2 Reaching the web interface . . . 42
14.3 Observing . . . 42
14.4 Safety features . . . 42

15 Troubleshooting **43**
15.1 System does not start . . . 43
15.2 Motors do not move . . . 43
15.3 Position errors . . . 43
15.4 Pointing errors . . . 43
15.5 Network and web interface . . . 44
15.6 Coordinates and Stellarium . . . 44

16 Outstanding work and known limitations **44**
16.1 Known limitations . . . 44
16.2 Where the open work is recorded . . . 45

17 The 21 cm line: expected signals and the temperature scale **45**
17.1 The hyperfine transition . . . 45
17.2 What the line measures . . . 45
17.3 Galactic rotation and the tangent-point method . . . 46
17.4 What the telescope collects . . . 47
17.5 Expected antenna temperatures . . . 47
17.6 Why only the Sun and Moon can calibrate a 5° beam . . . 48
17.7 Expected system temperature . . . 49
17.8 Measuring the system temperature . . . 49
17.8.1 Method A: ambient absorber . . . 49
17.8.2 Method B: the Sun, of known flux density . . . 49
17.8.3 The two-point calibration: blank sky and the Sun as loads . . . 50
17.8.4 Method C: cold sky and the survey slope . . . 52
17.8.5 Recommended procedure . . . 52

18 The codebase **54**
18.1 Repository and licence . . . 54
18.2 Working on GitHub . . . 54
18.3 Layout . . . 54
18.4 Root and drive firmware . . . 55
18.5 Controller firmware . . . 55
18.6 Receiver, scheduler and calibration . . . 56
18.7 Sky simulator . . . 56
18.8 Documentation, enclosure and tools . . . 57
18.9 What is deliberately not tracked . . . 58

19 Conclusion **58**

A Pin quick reference **59**

B Command quick reference **59**

C HTTP API quick reference **59**
C.1 Controller API 59
C.1.1 Status and information 59
C.1.2 Telescope control 59
C.1.3 Pointing, time, network and settings 60
C.2 Scheduler API 60
C.3 Examples 61

Index **63**

1 Introduction

1.1 Purpose and scope

The SRT drive controller is a complete control system for an alt-azimuth mounted radio telescope observing at 1420.405 MHz. It supports:

- Real-time telescope positioning from a web interface
- Integration with planetarium software (Stellarium)
- Conversion between celestial and horizontal coordinates, with atmospheric refraction and a fitted mount pointing model applied
- Measurement of the pointing model from raster scans of the Sun
- Scheduled observations with automatic data acquisition
- Sun, Moon, galactic-plane and satellite tracking

This document describes the system as it now stands. Where a design decision is non-obvious the reasoning is given, because several of them were reached the hard way and are easy to undo by accident.

1.2 System overview

The system separates concerns across four processing units and one host computer:

1. **Arduino Due** — low-level motor control, encoder counting, limit-switch homing, current sensing and fault handling. It works exclusively in *drive* coordinates.
2. **WT32-ETH01 controller** — network interface, astronomical transforms, the resident pointing model, web server and Stellarium protocol. It is the boundary between the sky and the mount.
3. **H1 receiver** — GNU Radio spectrometer for 21 cm data acquisition.
4. **Observation scheduler** — Flask application coordinating pointing and recording, and hosting the Sun-scan calibration.
5. **Observatory computer** — runs the receiver and the scheduler, and owns a private Ethernet link to the controller on which it provides the address, DNS and a route to the internet.

A fifth component, the **sky simulator** (Section 11), is not part of the observing chain but predicts what the instrument should see and can hand a simulated pointing to the real telescope.

1.3 Two coordinate frames

One distinction runs through the whole system and is worth stating before anything else.

True alt/az
: Where the dish looks on the sky. Every astronomical quantity is computed in this frame, using the true site latitude and longitude.

Drive
: What the mount mechanically is: encoder pulses counted from the limit switches, quantised to 0.5°. The Arduino Due works only in this frame.

The two are joined by the transform in Section 7, which applies atmospheric refraction and the fitted pointing model. Every commanded position crosses from true to drive on its way out; every *measured* position crosses from drive to true on its way back. Arrival and slew-completion tests are made wholly within the drive frame, so the two never mix.

1.4 Document organisation

Section 2 presents the system architecture; Section 3 the hardware; Section 4 the Arduino Due firmware; Section 5 the ESP32 controller; Section 6 the coordinate transformations; Section 7 the pointing model and its calibration; Section 8 the observatory host and the private controller link; Section 9 the beam, the radiometry and the velocity frames; Section 10 the receiver and scheduler; Section 11 the sky simulator; Section 12 installation; Section 13 testing and verification; Section 14 operation; Section 15 fault-finding; Section 16 the work still open; Section 17 the astronomy of the 21 cm line, the signal strengths this telescope should see, and how to measure its system temperature from the sky and the Sun; and Section 18 the repository and the purpose of every file in it.

2 System architecture

2.1 Architectural overview

Figure 1 shows the components and the paths between them.

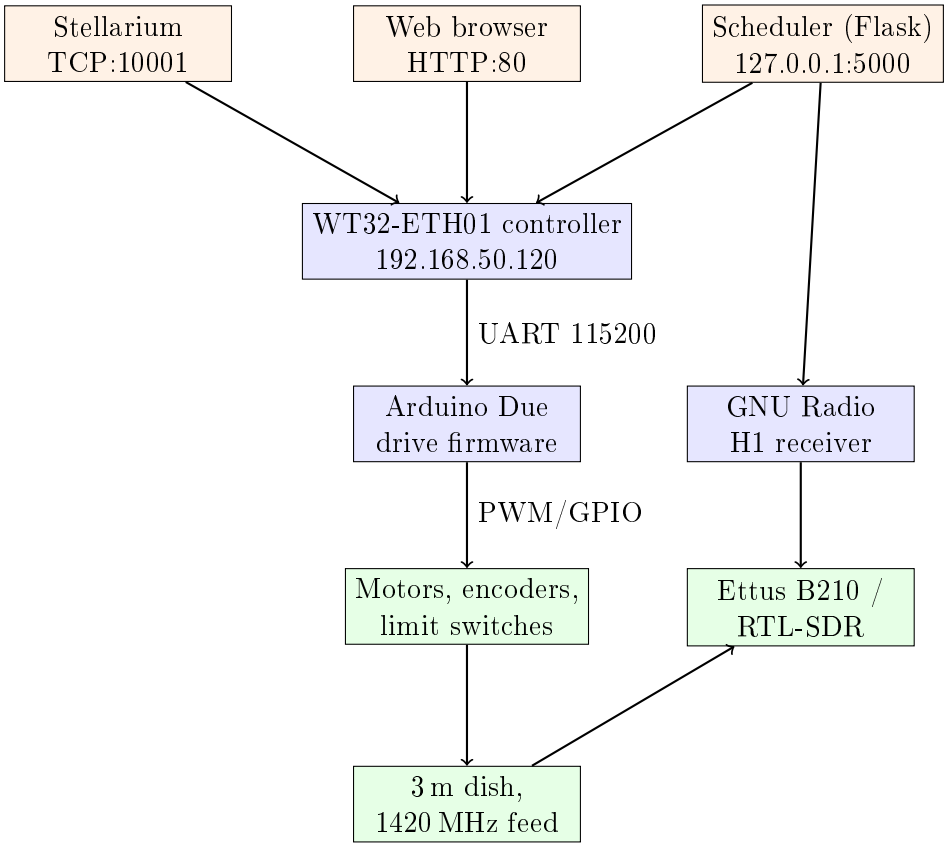

Figure 1: System architecture. Stellarium, the browser and the scheduler all run on the observatory computer, which reaches the controller over the private Ethernet link of Section 8.

2.2 Data flow

A typical observation proceeds as follows:

1. The user or scheduler specifies a target (RA/Dec, galactic, alt/az, solar-system object, or TLE).
2. The controller converts the target to *true* alt/az for the current sidereal time.
3. The observing horizon is tested against the true altitude.
4. `trueToDrive()` applies refraction and the pointing model, producing a *drive* position.
5. The drive position is tested and clamped against the mechanical mount limits, then sent to the Due over UART.
6. The Due computes a motion profile and drives the motors under PWM; reed-switch encoders count position.
7. The Due reports drive position, motor currents and status once a second.
8. The controller publishes the drive position as it stands, and its `driveToTrue()` image as the sky position, from which RA/Dec and galactic coordinates are derived.
9. Once the slew has completed, the scheduler starts the receiver and data is written to HDF5.

2.3 Communication interfaces

Table 1: System communication interfaces

Interface	Protocol	Speed	Purpose
Due ↔ controller	UART (Serial1)	115200 baud	Drive targets, status
Due ↔ USB	Serial	115200 baud	Debug, direct control
Controller ↔ browser	HTTP	TCP:80	Web interface, REST API
Controller ↔ Stellarium	TCP	Port 10001	Telescope protocol
Controller ↔ scheduler	HTTP	TCP:80	Pointing, status, model push
Controller Ethernet	100BASE-TX	LAN8720 RMII	Private link to the host
Controller WiFi	IEEE 802.11	—	AP fallback at 192.168.4.1
Controller OTA	ArduinoOTA	UDP/TCP 3232	Firmware update over Ethernet
Scheduler ↔ browser	HTTP	127.0.0.1:5000	Scheduler UI and API

The scheduler binds to loopback only. It has no authentication and its endpoints start observations, claim the SDR, rewrite the schedule, push a pointing model and flash controller firmware; exposed on a wildcard address it would be an unauthenticated telescope-control API. Nothing needs the wildcard — the controller's own page fetches the scheduler at `127.0.0.1:5000` from a browser on the same host, and remote use is over `waypipe`.

3 Hardware configuration

3.1 Arduino Due microcontroller

The Arduino Due is the real-time motor control platform, chosen for its ARM Cortex-M3 core at 84 MHz with deterministic interrupt handling, its 12-bit ADCs, multiple hardware UARTs, adequate PWM resolution and 3.3 V logic compatible with the ESP32.

3.1.1 Pin assignments

Table 2: Arduino Due pin assignments

Function	Pin	Wire colour	Notes
Fault flags (inputs)			
Az fault flag 1	5	Blue	Driver status
Az fault flag 2	4	Purple	Driver status
Alt fault flag 1	7	Blue	Driver status
Alt fault flag 2	6	Purple	Driver status
Motor control (outputs)			
Az PWM	8	Green	Speed control (inverted)
Az direction	9	Yellow	Sense inverted in software
Alt PWM	10	Green	Speed control (inverted)
Alt direction	11	Yellow	Sense inverted in software
Driver reset (outputs)			
Az reset	22	Black	HIGH = enabled
Alt reset	24	White	HIGH = enabled
Position encoders (inputs)			
Az pulses	12	Yellow	Reed switch, falling edge
Alt pulses	13	Blue	Reed switch, falling edge
Current sensors (analogue inputs)			
Az current	A1	Black	Hall effect
Alt current	A0	White	Hall effect
Serial1 (controller interface)			
TX1	18	—	To controller RX (IO14)
RX1	19	—	From controller TX (IO4)
Calibrator (output)			
Calibrator control	26	—	Active HIGH, noise source

The direction sense of *both* axes is inverted in software, by the `AZ_DIR_INVERT` and `ALT_DIR_INVERT` macros in `include/config.h`. Every read or write of `PIN_DIR_AZ` and `PIN_DIR_ALT` must therefore go through the `AZ_DIR()` and `ALT_DIR()` macros; a direct `digitalWrite` to those pins will drive the wrong way.

3.2 Motor drive system

3.2.1 H-bridge motor drivers

Each axis is driven by an H-bridge module with:

- Two fault flag outputs (FF1, FF2) encoding the fault type (Table 9)
- An active-low reset input, held HIGH to enable the driver
- Inverted PWM speed control: 255 is stopped, 0 is full speed
- A single direction input

The fault outputs are electrically noisy at motor start and stop, because of inductive kick-back and supply sag. `checkFaultFlags()` therefore requires the same fault to be present on `FAULT_FLAG_PERSIST_COUNT` (5) consecutive calls before reporting it.

3.2.2 Position encoders

Position feedback comes from reed-switch encoders on each axis:

- Resolution `PULSES_PER_DEGREE` = 2, i.e. a 0.5° quantum
- Falling-edge interrupts; direction is inferred from the static direction pin at the moment of the pulse
- Debounce windows of 50 ms (azimuth) and 100 ms (altitude), both settable

The debounce timestamp is updated *only* on pulses that pass the window. Updating it on every edge — the original implementation — caused total pulse loss as the motor speed approached the debounce interval, because every bounce reset the timer and no edge ever counted. This must not be reintroduced.

3.2.3 Current sensing

Motor current is measured by hall-effect sensors read through the Due's 12-bit ADC against its 3.3 V reference. The sensors are bipolar with a mid-rail zero, so

$$V = \frac{V_{\text{ref}}}{2^{12}}\, N_{\text{ADC}}, \qquad I = \frac{V - V_{\text{offset}}}{S}, \tag{1}$$

with $V_{\text{ref}} = 3.3\,\text{V}$, a nominal $V_{\text{offset}} = 1.65\,\text{V}$ and sensitivity $S = 0.044\,\text{V/A}$.

Two refinements matter. First, the reported current is a single-pole low-pass filter of the raw reading,

$$I_k = I_{k-1} + \alpha\,(I_k^{\text{raw}} - I_{k-1}), \qquad \alpha = 0.02, \tag{2}$$

and it is the *filtered* value, never the raw one, that `checkCurrentLimits()` tests — otherwise a single ADC excursion trips an overcurrent fault. Second, the zero offset is re-tracked continuously while an axis is idle,

$$V_k^{\text{offset}} = V_{k-1}^{\text{offset}} + \beta\,(V_k - V_{k-1}^{\text{offset}}), \qquad \beta = 0.005, \tag{3}$$

so long-term drift of the hall sensor settles towards zero by itself. The offsets are additionally measured directly at startup, by averaging 50 samples with the motors off.

3.3 ESP32 controller

The WT32-ETH01 is an ESP32 board with a native LAN8720A Ethernet PHY on RMII as well as WiFi. It is the deployed board and the canonical build target.

3.3.1 WT32-ETH01 to Arduino Due wiring

IO4 and IO14 are used for the Due link, keeping TX0/RX0 free for the first serial flash and for recovery.

Table 3: WT32-ETH01 to Arduino Due serial connection

WT32-ETH01	Arduino Due	Function
IO4	Pin 19 (RX1)	Controller TX → Due RX
IO14	Pin 18 (TX1)	Controller RX ← Due TX
GND	GND	Common ground
5V	5V	Power

Table 4: WT32-ETH01 first-flash connection (temporary FT232 programmer)

FT232 programmer	WT32-ETH01	Function
TXD	RX0 (GPIO3)	Programmer TX → ESP32 RX
RXD	TX0 (GPIO1)	Programmer RX ← ESP32 TX
GND	GND	Common ground

The programmer is temporary: its power, IO0, EN, DTR and RTS wiring must not remain connected in normal operation. After one OTA-capable serial flash, routine updates go over Ethernet with the `wt32-eth01-ota` environment.

A printed enclosure for the board is in `enclosure/`, as an OpenSCAD source with the board dimensions and wall thicknesses as named parameters, plus exported STLs for the base and lid. It is designed to mount on the outside of a larger box with a 15 mm wire feedthrough, and the Ethernet jack sets the internal height.

3.4 Operational limits

Table 5: Position limits. The hardware limits are the physical switches; the software limits are the operating envelope inside them.

Axis	Minimum	Maximum
Altitude (hardware)	0° (limit switch)	90° (zenith)
Altitude (software)	0°	90°
Azimuth (hardware)	0° (limit switch)	355° (limit switch)
Azimuth (software)	2°	353°

The software azimuth limits sit 2° inside the physical stops. A further `SOFTWARE_LIMIT_TOLERANCE` of 4° is allowed as a transient excursion before the firmware forces a stop, but never past a hardware limit.

These are all *drive*-frame quantities. The separate observing horizon (Section 5) is a true-frame quantity and is a different thing entirely.

4 Arduino Due firmware

4.1 Firmware overview

The drive firmware (`src/main.cpp`) is a single bare-metal translation unit implementing:

- An automatic three-phase homing sequence at power-on
- Ramped motion profiles that prevent structural oscillation

- Interrupt-driven encoder position tracking
- Safety monitoring: overcurrent, stall, limit switches, driver fault flags
- A line-oriented serial command interface on both ports
- Configuration persistence to flash via DueFlashStorage

Position is held in encoder pulses. Displayed degrees are position/`PULSES_PER_DEGREE`, and pulse position 0 is the hardware lower limit on each axis. After homing, both axes read $(0, 0)$.

4.2 State machine

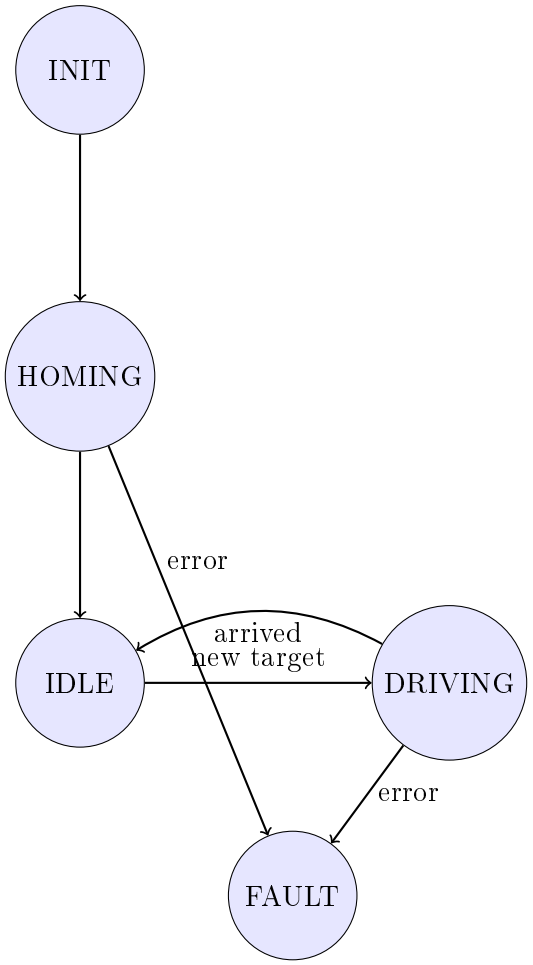

Figure 2: Drive firmware state machine

INIT Hardware initialisation, GPIO configuration, current-sensor zeroing, driver enable. Reported as `Initializing`.

HOMING Running the homing sequence. Reported as `Homing`.

IDLE Accepting commands. Reported as `Ready`.

DRIVING Slewing. Reported as `Slewing`, or `Reversing` while an axis is decelerating to change direction.

FAULT Motors disabled; reported as `FAULT` with the fault text in brackets.

The motion controller `updateAxisMotion()` runs from the main loop except in HOMING and FAULT. The IDLE→DRIVING transition fires *only* when a new-target flag has been set by `executeDrive()`. Without that condition the controller would chase stray encoder pulses while idle and could walk itself into a limit switch.

4.3 Homing sequence

Homing establishes the encoder origin in three phases, because a single fast run into the stop leaves an origin that depends on how fast the axis was travelling when it hit.

1. **Drive to limits.** Both axes ramp up to full speed and run negative into the hard stops. An axis is judged to have arrived when no encoder pulse has been seen for `stallTimeoutMs` (2 s by default).
2. **Back off.** Both axes reverse 5° under the same ramp profile.
3. **Re-approach.** Both axes run into the stops again over the short remaining distance, so the approach is slow and the zero is repeatable.

Both helpers obtain their profile from `calculatePWM(INT32_MAX, startTime)`, which yields the standard ramp-up without ever entering the ramp-down branch. After phase 3 both position counters are set to zero. The mount then drives to the configured home offset,

$$n_{\text{home}} = \texttt{cfg.homeAz} \times \texttt{PULSES\_PER\_DEGREE}, \tag{4}$$

computed directly from the home angle — the software minimum is *not* subtracted — and the position counters are set to the home coordinates. Both `homeAlt` and `homeAz` default to 0°, so by default the mount finishes homing sitting on its origin. A fault flag, overcurrent condition or unresponsive motor at any point aborts homing into FAULT. The whole sequence takes 30–60 s depending on the starting position.

4.4 Motion profiles

PWM values are inverted: PWM = 255 is stopped, PWM = 0 is full speed, and `PWM_MIN_SPEED` = 100 is the slowest usable drive. In the expressions below a *smaller* number is a faster motor.

4.4.1 Acceleration ramp

For the first T_{ramp} milliseconds of a slew (`RAMPUP`, default 500 ms) the PWM value interpolates linearly from the minimum speed to full speed:

$$\text{PWM}(t) = \text{PWM}_{\text{min}} - \frac{t}{T_{\text{ramp}}}\big(\text{PWM}_{\text{min}} - \text{PWM}_{\text{full}}\big) = 100\left(1 - \frac{t}{T_{\text{ramp}}}\right). \tag{5}$$

Thereafter the axis cruises at $\text{PWM}_{\text{full}} = 0$.

4.4.2 Deceleration ramp

Deceleration is keyed to the number of encoder pulses remaining, n, and begins when n falls below $n_{\text{ramp}} = \texttt{RAMPDOWN} \times \texttt{PULSES\_PER\_DEGREE}$ (default 7°, so 14 pulses):

$$\text{PWM}(n) = \min\big(\text{PWM}_{\text{min}},\ \max(\text{PWM}_{\text{full}},\ 218 - n^2)\big). \tag{6}$$

The quadratic in n gives an approximately constant deceleration in position; the clamp at PWM_{min} stops the axis creeping so slowly in the last degree that the encoder debounce swallows its pulses. `STOPRAMP` (default 300 ms) is the corresponding deceleration time applied before an axis reverses direction.

4.5 Stall and limit handling

A stall is declared only when *both*

$$t - t_{\text{drive start}} > T_{\text{stall}} \qquad \text{and} \qquad t - t_{\text{last pulse}} > T_{\text{stall}} \tag{7}$$

are satisfied. The first condition matters: without it a stale `lastPulse` left over from a long idle period would trigger a false stall the instant a new slew began.

A stall that occurs while driving negative and within 2° of the hardware limit is not a fault — it is the limit switch. The position snaps to the limit value, the target is clamped, and the axis returns to idle. This is how the mount recovers gracefully from a commanded position slightly outside its range.

4.6 Serial command interface

Commands are accepted on the programming-port USB serial and on Serial1 (the controller link), both at 115200 baud.

Table 6: Motion and calibrator commands

Command	Description
`<alt> <az>`	Slew to position (e.g. `45.0 180.0`)
`DRIVE <alt> <az>`	Slew to position (explicit form)
`HOME`	Run the homing sequence
`STOP`	Emergency stop
`RESET`	Clear fault
`CAL ON` / `CAL OFF` / `CAL`	Noise-source calibrator on, off, toggle

The calibrator is a noise diode used for receiver calibration; `CAL ON` drives GPIO 26 high.

Table 7: Configuration commands

Command	Description
`STATUS`	Print one status line immediately
`CONFIG`	Show all configuration parameters
`SET <param> <value>`	Set a configuration parameter
`SAVE`	Save configuration to flash
`LOAD`	Load configuration from flash
`DEFAULTS`	Reset to compiled defaults
`HELP`	Show command help

4.6.1 Configurable parameters

Table 8: `SET` parameters

Parameter	Description	Default	Unit
ALTHWMIN, ALTHWMAX	Altitude hardware limits	0, 90	degrees
AZHWMIN, AZHWMAX	Azimuth hardware limits	0, 355	degrees
ALTMIN, ALTMAX	Altitude software limits	0, 90	degrees
AZMIN, AZMAX	Azimuth software limits	2, 353	degrees
HOMEALT, HOMEAZ	Home position (encoder origin)	0, 0	degrees
RAMPUP	Acceleration time	500	ms
RAMPDOWN	Deceleration distance	7	degrees
STOPRAMP	Reversal deceleration time	300	ms
CURRENT	Overcurrent threshold	5.0	A
STALL	Stall detection timeout	2000	ms
DEBOUNCE	Azimuth encoder debounce	50	ms
DEBOUNCE_ALT	Altitude encoder debounce	100	ms
BACKLASH	Azimuth backlash compensation	0.0	degrees

Configuration lives in a `Config` struct in RAM and is persisted to flash on `SAVE`. `loadDefaults()` must initialise *every* field — a field left out holds garbage until the next `SAVE`. Adding a tunable parameter means five coordinated edits: a `DEFAULT_X` macro in `include/config.h`, a field in the `Config` struct, an initialisation in `loadDefaults()`, a case in `processSetCommand()`, and lines in `showConfig()` and `showHelp()`.

4.7 Status output format

The status line is positional and the controller's parser depends on it. Do not change the field order or spelling.

```
Alt:%.1f Az:%.1f Ialt:%.1fA Iaz:%.1fA Status:<state> [<fault>] -> Alt:%.1f Az:%.1f Cal:ON|OFF
```

The `[<fault>]` group appears only in FAULT, and the `-> Alt: Az:` group only while DRIVING.

```
1 Alt:45.0 Az:180.0 Ialt:0.5A Iaz:0.4A Status:Ready Cal:OFF
2 Alt:30.0 Az:150.0 Ialt:2.1A Iaz:1.9A Status:Slewing -> Alt:45.0 Az:180.0 Cal:OFF
3 Alt:30.0 Az:150.0 Ialt:5.2A Iaz:0.0A Status:FAULT [Altitude motor overcurrent]
    Cal:OFF
```

Listing 1: Status output examples

Two independent output disciplines apply.

Programming-port de-duplication
: `outputStatus()` builds a comparison string that excludes the live current readings and prints to the USB port only when that string changes. Without it the port would emit a line per main-loop pass with nothing but ADC noise varying.

Serial1 rate limiting
: The unsolicited line to the controller is emitted at most once per `STATUS_INTERVAL_MS` (1 s). It used to be unconditional, which meant roughly 100 lines per second, about 6.5 kB/s, into a receiver that drains five lines a second through a 256-byte buffer. That buffer was in permanent overflow, so bytes were dropped mid-line and the tail of one line arrived

welded to the head of the next — splices that still passed the controller's format check and parsed into plausible-looking nonsense. A `STATUS` command arriving on Serial1 is answered immediately on its own path and is not rate limited, and that is how the controller actually polls.

A `STATUS` command received on Serial1 is answered only on Serial1, and never disturbs the programming-port de-duplication buffer.

4.8 Fault detection

Table 9: Fault conditions and detection

Fault	Detection	Likely cause
Short circuit	FF1 LOW, FF2 HIGH	Wiring fault, motor failure
Overheating	FF1 HIGH, FF2 LOW	Prolonged high current
Undervoltage	FF1 HIGH, FF2 HIGH	Power supply sag
Overcurrent	Filtered $I >$ `CURRENT`	Obstruction, binding
Stall	No pulses for `STALL`	Motor failure, jam, limit
Position bounds	Position outside limits	Encoder error

4.9 Simulation mode

The `simulation` environment overrides `analogWrite`, `digitalWrite`, `analogRead` and `attachInterrupt` with macros, so the firmware runs end to end with no hardware attached. `simulatePulses()` advances position from the commanded PWM and is called both from the main loop and from inside the homing while-loops.

```
~/.platformio/penv/Scripts/pio.exe run -e simulation
```

Any change to the homing flow or to in-loop motor control must keep working in simulation; both the `due` and `simulation` environments must build clean.

Table 10: Simulation parameters (`include/config.h`)

Parameter	Default	Description
`SIM_MAX_SPEED_DEG_S`	180.0	Maximum simulated speed (deg/s)
`SIM_INITIAL_AZ_DEG`	20.0	Starting azimuth before homing
`SIM_INITIAL_ALT_DEG`	10.0	Starting altitude before homing
`SIM_STALL_TIMEOUT_MS`	200	Faster stall detection in simulation

The current-sensor pins are *not* stubbed: A0 and A1 read the real ADC even in simulation, so current sensing and its calibration can be exercised against real sensors while the motion loop runs simulated.

5 ESP32 controller

5.1 Software architecture

The controller is an Arduino-framework PlatformIO project. `state.h` declares the single shared `SRTState` struct; all subsystems mutate it directly, under a cross-task lock where the mutating task is not `loopTask`.

Table 11: Controller source files

File	Purpose
`main.cpp`	Application, tracking loop, clock discipline
`config.h`	Compile-time defaults
`settings.cpp/h`	Runtime settings, NVS persistence
`coordinates.cpp/h`	Astronomical transforms
`pointing.cpp/h`	Pointing model, true/drive transform, refraction
`web_server.cpp/h`	Async HTTP server and REST API
`stellarium.cpp/h`	Stellarium telescope protocol
`srt_serial.cpp/h`	UART link to the Due
`wifi_manager.cpp/h`	WiFi AP/station management
`index_html.h`	Embedded web interface

HTTP is served by ESPAsyncWebServer and the Stellarium protocol by AsyncTCP, both on the `async_tcp` task; the tracking loop runs on `loopTask`. State touched by both is guarded by the `SRTLock` scoped lock. Flash writes — settings and the pointing model — are deliberately performed *outside* that lock: they take tens of milliseconds and nothing on `loopTask` writes either structure, so there is no writer to race and no reason to stall tracking.

5.2 Settings

Table 12: Runtime settings and their compiled defaults

Setting	Default	Frame	Meaning
`observer_lat`	55.902426	—	True site latitude (deg N)
`observer_lon`	−4.307865	—	True site longitude (deg E)
`mount_alt_min/max`	0, 90	drive	Mechanical altitude stops
`mount_az_min/max`	2, 353	drive	Mechanical azimuth stops
`horizon_alt`	10.0	true	Local observing horizon
`galactic_min_alt`	45.0	true	Galactic-plane acquisition floor
`stow_alt`, `stow_az`	90, 180	drive	Park position
`position_deadband`	0.25	drive	Minimum move worth commanding
`ap_ssid`	`SRT_Controller`	—	WiFi AP name
`ap_password`	`radio1420`	—	WiFi AP password
`page_name`	—	—	Title shown in the web UI

Three distinctions in that table are load-bearing and have each been merged by mistake at least once.

Observing horizon versus mount limits

`horizon_alt` is a question about the sky — has the source cleared the trees — so it is tested against the target's *true* altitude, before the pointing transform. `mount_alt_min` is a question about the machine, and clamps the *drive* altitude afterwards. They were once merged into a single "effective tracking horizon" equal to the larger of the two, which quietly reinterpreted a mechanical stop as a sky horizon and left neither testable on its own.

Stow versus the Due's home

`stow_alt`/`stow_az` is where the dish *parks*. It is the one sky-looking setting held in *drive* coordinates, and the pointing model is deliberately bypassed on the stow path. Parking is mechanical — "leave the mount here" — not an observation, and at the default zenith stow

the distinction is not academic: azimuth is degenerate there, every azimuth points at the same piece of sky, and the model's $\tan h$ azimuth term asks for a correction of order 50° that would move the beam by exactly nothing. Through the model the dish would park at drive azimuth 170°. The Due's `HOMEALT`/`HOMEAZ` are also drive coordinates but mean something different again: the *encoder origin*, what the limit-switch stall corresponds to.

Operator offset versus calibration
The `/offset` boxes are a deliberate sky-frame displacement layered on top of the calibration — for raster mapping and beam switching — and never a substitute for it. The offset is applied in the true frame, before the model, so "0.5° off source" means half a degree on the sky at any elevation.

5.3 Networking

5.3.1 Network modes

1. **Ethernet** — LAN8720 PHY, DHCP by default. This is the operational path; see Section 8.
2. **WiFi access point** — always active at 192.168.4.1, independent of the Ethernet configuration, and the standing fallback.
3. **WiFi station** — connects to a saved network if one is configured.
4. **mDNS and OTA** — discoverable as `srt-controller.local`; firmware upload over Ethernet on port 3232.

Table 13: Default network configuration

Setting	Value
Ethernet address	DHCP (pinned by MAC to 192.168.50.120)
Static-IP fallback	192.168.50.120/24, gateway and DNS 192.168.50.1
WiFi AP SSID / password	`SRT_Controller` / `radio1420`
AP address	192.168.4.1
Hostname	`srt-controller.local`
Web server port	80
Stellarium port	10001
OTA port	3232
NTP server	`pool.ntp.org`, numeric fallback 162.159.200.123

The static-IP values are never applied in normal operation, because the controller runs on DHCP — but they are what the Static IP form is pre-filled with, so they must name the link the controller is actually on. They were once left at a `192.168.1.x` factory placeholder, which meant one tick of that box would have moved the controller off the link entirely, reachable only over the WiFi AP.

5.3.2 Clock discipline

The lwIP SNTP client polls in the background every hour; nothing blocks waiting for it. Success is declared only by the `onTimeSync()` callback, so a clock that has never been set cannot report itself as synced. `/time/status` publishes `sync_state`, `stale`, `last_sync_age_s` (which is −1, not 0, when no sync has ever happened), `last_offset_ms` and `sync_count`. A browser-set time is recorded as source `browser` and deliberately does *not* update the last-sync epoch, so it reports as unverified rather than masquerading as a checked clock.

Clock accuracy is a pointing specification, not a convenience. An error of τ seconds in UTC appears directly as an hour-angle error of $15.041\,\tau$ arcseconds, so the mount loses half an encoder quantum (0.25°) after only 60 s of clock error, and a degree after four minutes.

5.3.3 DNS resilience

lwIP keeps one process-wide resolver list rather than one per interface, and bringing the WiFi AP up clears it. The firmware caches the resolver address supplied by DHCP and restores it whenever it finds the list empty. `/network` reports `dns_after_eth`, `dns_after_wifi`, `lwip_dns0` and a `dns_restores` counter; `dns_after_wifi` reading `0.0.0.0` and a slowly climbing restore count are both expected, not faults.

5.4 HTTP API

The full endpoint list is in Appendix C. The endpoints that carry frame semantics are worth stating here.

`/status`
: `alt` and `az` are the Due's *drive* position — the scheduler's slew-completion check and `sun_scan.py`'s record of where a source was found both need that frame, and both compare against a drive target. `true_alt` and `true_az` are the sky position it corresponds to, and it is those, not the drive position, from which RA/Dec and galactic l, b are computed. Converting the drive position straight to RA/Dec would be wrong by the whole calibration. The response also carries `pointing_loaded`, `clock_state`, `clock_age_s`, and the `lock_timeouts` and `malformed_status` counters, both of which are zero in normal operation.

`/goto`, `/track/*`, `/direct`
: Targets are true-frame and are rejected below `horizon_alt` before any state is mutated. They pass through `trueToDrive()` before being sent to the Due.

`/go-home`
: Slews to the stow position. This is the documented exception: stow is already a drive position, so it is sent as it stands and merely clamped to the mount limits. `state.targetAlt`/`targetAz` are deliberately *not* written, because those hold a true-frame target and putting drive coordinates in them would be exactly the frame mix the design exists to prevent. `/home`, by contrast, runs the Due's physical homing sequence.

`/pointing`, `/pointing/apply`, `/pointing/clear`
: The resident pointing model; see Section 7.

`/ping`, `/network`
: Minimal diagnostics, registered ahead of the full UI handler so they answer even if the main page does not.

5.5 Stellarium protocol

The controller implements the Stellarium telescope protocol on TCP port 10001: a 20-byte goto message carrying RA and Dec as unsigned 32-bit fractions, and a 24-byte position report in the same encoding. Reported positions are `driveToTrue()` images of the measured drive position, so the marker in Stellarium is where the dish is looking rather than where the encoders read.

To configure: Configuration → Plugins → Telescope Control, enable and restart, add a telescope of type "External software or remote computer", host `192.168.50.120`, port 10001. Select an object and press Ctrl+1 to slew.

5.6 Tracking

With tracking enabled the controller, once a second:

1. Refreshes the target's RA/Dec (every 30 s for the Sun, Moon and galactic plane; static otherwise)
2. Converts to true alt/az for the current sidereal time
3. Tests the true altitude against `horizon_alt`
4. Applies the operator offset and any axis-hold, in the true frame
5. Converts to the drive frame with `trueToDrive()`
6. Tests and clamps against the mount limits
7. Sends the drive target if it differs from the last one by more than the deadband

5.6.1 Below-horizon handling

When the true altitude falls below `horizon_alt` the dish parks at the stow position and the controller enters "waiting for rise". Position calculation continues each second, and tracking resumes automatically when the target clears the horizon again.

5.6.2 Azimuth wrap-around and the upper stop

If the drive azimuth falls outside the mount limits — which happens to circumpolar sources crossing the azimuth dead zone — the controller enters "waiting for wrap", continues computing, and resumes when the target re-enters the range. Note that the rise verdict is settled *before* the transform and the wrap verdict *after* it: clearing the wrap flag in the true frame would announce "wrap complete" every second for a target still mechanically unreachable.

Above `mount_alt_max` the mount is simply left where it is. The alternative — driving to the altitude stop and continuing to track in azimuth — would run the mount along its limit switch for hours to follow a target it cannot point at.

5.6.3 Galactic-plane target

`/track/galactic-plane` walks outwards along $b = 0$ from the galactic centre in 0.5° steps and returns the first longitude whose current altitude reaches `galactic_min_alt`. At each step the two candidates $l = +d$ and $l = 360° - d$ are equidistant from the centre, so the choice between them is free and the higher is taken — it stays observable longer and looks through less atmosphere.

The 45° default floor is an *acquisition* threshold, not a horizon: once acquired the target is followed down until the ordinary observing horizon parks the dish. It is high deliberately. The galactic centre is at $\delta = -28.9°$ and from Glasgow ($\phi = 55.90°$) culminates at

$$h_{\max} = 90° - |\phi - \delta| = 90° - 84.8° = 5.2°, \tag{8}$$

so it never clears the 10° horizon — zero hours in a year. A fixed "galactic bulge" button was therefore permanently in fallback. At a 45° floor some point on the plane qualifies about 73% of the time, and never within about 45° of longitude of the centre.

6 Coordinate transformations

6.1 Reference frame

All equatorial coordinates are J2000.0: standard epoch and equinox J2000.0, equivalent to ICRS to within milliarcseconds, and compatible with modern catalogues and with Stellarium.

6.2 Conversion pipeline

$$\text{RA/Dec (J2000)} \xrightarrow{\text{precess}} \text{RA/Dec (date)} \xrightarrow{\text{HA}} \text{true Alt/Az} \xrightarrow[+\ \text{model}]{+\ \text{refraction}} \text{drive Alt/Az} \tag{9}$$

The first three steps are described here; the last is Section 7.

6.2.1 Julian date

$$\text{JD} = \lfloor 365.25(Y+4716)\rfloor + \lfloor 30.6001(M+1)\rfloor + D + B - 1524.5 + \frac{h + m/60 + s/3600}{24} \tag{10}$$

with Y the year, M the month (January and February counted as months 13 and 14 of the previous year), D the day, $B = 2 - \lfloor Y/100\rfloor + \lfloor Y/400\rfloor$, and h, m, s UTC.

6.2.2 Sidereal time

GMST in hours, by the IAU 1982 expression [3]:

$$\text{GMST} = 6.697374558 + 0.06570982441908\,D_0 + 1.00273790935\,H + 2.6\times10^{-5}\,T^2 \tag{11}$$

where D_0 is days since J2000.0 at 0^{h} UT, H is hours since 0^{h} UT, and $T = D_0/36525$. Local sidereal time follows as

$$\text{LST} = \text{GMST} + \lambda/15 \tag{12}$$

with λ the observer's longitude in degrees, east positive.

6.3 Precession

IAU 1976 precession [2] takes J2000 coordinates to the equinox of date. With $T = (\text{JD} - 2451545.0)/36525$ the angles, in arcseconds, are

$$\zeta_A = (2306.2181 + 1.39656\,T)\,T + 0.30188\,T^2 + 0.017998\,T^3 \tag{13}$$
$$z_A = (2306.2181 + 1.39656\,T)\,T + 1.09468\,T^2 + 0.018203\,T^3 \tag{14}$$
$$\theta_A = (2004.3109 - 0.85330\,T)\,T - 0.42665\,T^2 - 0.041833\,T^3 \tag{15}$$

and the rotation is applied through

$$A = \cos\delta_0 \sin(\alpha_0 + \zeta_A) \tag{16}$$
$$B = \cos\theta_A \cos\delta_0 \cos(\alpha_0 + \zeta_A) - \sin\theta_A \sin\delta_0 \tag{17}$$
$$C = \sin\theta_A \cos\delta_0 \cos(\alpha_0 + \zeta_A) + \cos\theta_A \sin\delta_0 \tag{18}$$

giving

$$\alpha = \operatorname{atan2}(A, B) + z_A, \qquad \delta = \arcsin C. \tag{19}$$

6.4 Horizontal coordinates

With hour angle $\mathrm{HA} = \mathrm{LST} - \alpha$ and observer latitude ϕ,

$$\sin h = \sin\delta \sin\phi + \cos\delta \cos\phi \cos \mathrm{HA}, \tag{20}$$

$$\cos A = \frac{\sin\delta - \sin h \sin\phi}{\cos h \cos\phi}, \qquad A = \begin{cases} \arccos(\cos A) & \sin \mathrm{HA} \le 0 \\ 2\pi - \arccos(\cos A) & \sin \mathrm{HA} > 0. \end{cases} \tag{21}$$

Azimuth is measured from north through east. This yields the *true* alt/az of Section 1.3.

6.5 Galactic coordinates

Using the J2000 galactic pole constants $\alpha_{\mathrm{NGP}} = 192.85948°$, $\delta_{\mathrm{NGP}} = 27.12825°$ and $l_{\mathrm{NCP}} = 122.93192°$,

$$\sin\delta = \sin\delta_{\mathrm{NGP}} \sin b + \cos\delta_{\mathrm{NGP}} \cos b \cos(l_{\mathrm{NCP}} - l) \tag{22}$$

$$\alpha = \alpha_{\mathrm{NGP}} + \mathrm{atan2}\big(\cos b \sin(l_{\mathrm{NCP}} - l),\ \cos\delta_{\mathrm{NGP}} \sin b - \sin\delta_{\mathrm{NGP}} \cos b \cos(l_{\mathrm{NCP}} - l)\big) \tag{23}$$

with the inverse used for the l, b published in `/status`.

6.6 Solar position

Following Meeus [1], with T in Julian centuries from J2000.0:

$$L_0 = 280.46646 + 36000.76983\,T \quad \text{(mean longitude)} \tag{24}$$

$$M = 357.52911 + 35999.05029\,T \quad \text{(mean anomaly)} \tag{25}$$

$$C = 1.914602 \sin M + 0.019993 \sin 2M + 0.000289 \sin 3M \quad \text{(equation of centre)} \tag{26}$$

$$\lambda_\odot = L_0 + C \quad \text{(true longitude)} \tag{27}$$

followed by conversion from ecliptic to equatorial coordinates and precession from the equinox of date back to J2000. Accuracy is about 1 arcminute — entirely adequate against a 3° beam, and the reason the Sun is a usable pointing calibrator.

6.7 Lunar position

A truncated ELP/MPP02: mean elements L', D, M, M', F, the principal periodic terms in longitude and latitude, geocentric ecliptic to equatorial, then precession to J2000. Accuracy 5–10 arcminutes, comfortably inside the 0.5° encoder quantum.

6.8 Accuracy of the transforms

Compared against ERFA/astropy the transforms above agree to better than 0.5 arcminute RMS in ordinary observing conditions. Residual error is dominated by, in order: the encoder quantum (0.5°), the mount pointing model residual (Section 7), clock error, and only then the transforms themselves. Azimuth becomes ill-conditioned near the zenith, where it is also physically degenerate.

The coordinate and pointing code is exercised on the host by the `native` PlatformIO environment, which compiles `coordinates.cpp` and `pointing.cpp` exactly as they are flashed — a small `test/shim/` supplies the little of `Arduino.h` and `Preferences.h` that `pointing.cpp` touches — rather than testing a copy.

7 Pointing model and calibration

Everything in Section 6 produces a position on the sky. This section is about the difference between that and where the mount must actually be driven to see it. The transform lives in `esp32_controller_arduino/src/pointing.{h,cpp}`; the measurement of its parameters lives in `receiver_scheduler/sun_scan.py`.

7.1 Atmospheric refraction

Refraction is applied whether or not a pointing model is loaded, because it is physics rather than calibration. The controller uses Bennett's formula [4] scaled for radio wavelengths:

$$R(h) = \frac{1.15}{60\,\tan\!\left(h + \dfrac{7.31}{h + 4.4}\right)} \text{ degrees}, \tag{28}$$

with h the geometric altitude in degrees. The factor 1.15 accounts for radio refractivity being about 315 N-units against 273 optical, the difference being the water-vapour wet term. A nominal atmosphere is assumed: there is no weather station, and the dry/wet swing is well below the residual of a single calibration scan. The altitude is clipped to $[-1°, 90°]$, and the result forced to zero past the zenith where the tangent's argument exceeds 90° and turns negative.

Equation (28) is duplicated as `refraction_deg()` in `sun_scan.py` and the two must stay identical. The controller adds refraction to every sky position it converts, so a scan's measured error contains it; the fit removes it before solving, and if it did not, the fitted terms would absorb refraction and the controller would then apply it a second time. At 11° elevation that double count is about 0.09° — the same size as the effect the whole calibration is chasing.

7.2 The model

The mount error is described by the standard alt-az pointing terms [5]. Writing h for altitude and A for azimuth, the correction to be *added* to a true position to obtain the drive position is

$$\Delta h = \mathrm{IE} + \mathrm{AN}\cos A + \mathrm{AE}\sin A - \mathrm{TF}\cos h, \tag{29}$$

$$\Delta A = \mathrm{IA} + \left(\mathrm{AN}\sin A - \mathrm{AE}\cos A\right)\tan h + \mathrm{CA}\sec h + \mathrm{NPAE}\tan h. \tag{30}$$

Table 14: Pointing model terms

Term	Name	Physical meaning
IE	Elevation index error	Zero-point error of the altitude encoder
IA	Azimuth index error	Zero-point error of the azimuth encoder
AN	Azimuth axis tilt, north	Rotation axis tipped towards north; AN $\approx \delta\phi$
AE	Azimuth axis tilt, east	Rotation axis tipped towards east; AE $\approx \delta\lambda\cos\phi$
CA	Collimation error	Beam not perpendicular to the altitude axis
NPAE	Axis non-perpendicularity	Altitude axis not perpendicular to the azimuth axis
TF	Tube flexure	Gravitational droop of the structure

The azimuth signs are opposite to the altitude ones for a reason worth remembering: tipping the azimuth axis towards north raises a source in the north but swings a source in the *east*

clockwise. The design matrix used by the fit has been checked against numerically rotated horizon coordinates rather than derived on paper alone.

All seven slots always exist and default to zero, so the four-term model actually fitted from Sun data is simply one with CA, NPAE and TF left at zero. A later, richer fit needs no change to the storage, the transform or the wire format.

7.3 The forward and inverse transforms

`trueToDrive()` applies refraction first and evaluates the model at the *apparent* position:

$$h_{\text{app}} = h_{\text{true}} + R(h_{\text{true}}) \tag{31}$$

$$h_{\text{drive}} = h_{\text{app}} + \Delta h(h_{\text{app}}, A_{\text{true}}) \tag{32}$$

$$A_{\text{drive}} = A_{\text{true}} + \Delta A(h_{\text{app}}, A_{\text{true}}) \pmod{360^\circ} \tag{33}$$

`driveToTrue()` inverts this by fixed-point iteration *on the forward transform itself*, so that whatever the forward direction does, the inverse undoes exactly that and the two cannot drift apart. Starting from $(h^{(0)}, A^{(0)}) = (h_{\text{drive}}, A_{\text{drive}})$ and writing F for `trueToDrive`,

$$\big(h^{(k+1)}, A^{(k+1)}\big) = \big(h^{(k)}, A^{(k)}\big) + \Big[\big(h_{\text{drive}}, A_{\text{drive}}\big) - F\big(h^{(k)}, A^{(k)}\big)\Big], \tag{34}$$

with the azimuth residual taken the short way round modulo 360° so that a guess either side of due north does not chase itself through a full turn. Three iterations are used.

Simply negating the correction evaluated at the drive position is only first-order accurate. That is adequate for the four-term model, but with CA and NPAE present the azimuth correction carries $\sec h$ and $\tan h$, and near the zenith the first-order answer is wrong by about a quarter of a degree — half the encoder quantum, which is not a rounding error. Three passes bring the error to well under a thousandth of a degree anywhere the mount can point.

7.4 Guards

Two clamps protect the mount from a broken or mis-scaled model. The altitude fed to the $\tan h$ and $\sec h$ terms is limited to 89° — at that altitude $\tan h$ is already 57, far past anything a real term needs, and azimuth is degenerate there anyway. The total correction on each axis is then limited to ±10°; the whole measured error on this mount is around two degrees, so anything beyond that is not a pointing term. Clamping is silent by design, because the model is still reported by `/pointing` and a model that hits the clamp is therefore visible without a slew having to be commanded from it.

7.5 Where the calibration lives

The model is held in the controller's own NVS namespace, separate from the settings namespace on purpose: the calibration is a measurement, not a preference, and "reset settings to defaults" must not silently discard it. An absent or malformed record leaves the identity transform — never a guess.

This replaced an earlier scheme in which the model was smuggled to the controller as a fictitious observer latitude and longitude plus a write to the operator's pointing-offset boxes. That had two failure modes. The web UI reported a location the telescope is not at, and the offset half lived in RAM only, so every reboot silently discarded half the model: measured on 2026-08-19, the azimuth error that reappeared after a power cycle was 1.97° against a stored offset of 1.98°, about a quarter of the signal, lost without a word. The observer position is now the true site position and `/pointing/apply` does not touch it.

7.6 Measuring the model: the Sun scan

A single scan is an $n \times n$ raster (default 5×5 at $1.5°$ spacing) centred on the computed position of the Sun, measuring broadband power at each point.

7.6.1 Raster geometry

Offsets are specified on the sky. An altitude offset is applied directly; a cross-elevation offset Δx must be expanded into mount azimuth by the usual $\cos h$ factor:

$$h_{\mathrm{cmd}} = h_\odot + \Delta h, \qquad A_{\mathrm{cmd}} = A_\odot + \frac{\Delta x}{\cos h_\odot}, \tag{35}$$

clamped to the usable azimuth range, with the Sun's position recomputed before each point so the raster does not smear as the Sun moves through it. Scans are refused within about $0.6°$ of the zenith, where $\cos h_\odot$ makes the expansion meaningless.

7.6.2 Beam fit

A circular two-dimensional Gaussian is fitted to the measured power over the offset grid,

$$P(x, y) = a \exp\!\left(-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2}\right) + c, \tag{36}$$

whose centroid (x_0, y_0) is the pointing error of that scan and whose width gives the beam

$$\mathrm{FWHM} = 2\sqrt{2 \ln 2}\, \sigma \approx 2.3548\, \sigma. \tag{37}$$

The covariance returned by the fit supplies the per-scan centroid uncertainties, typically around $0.04°$, which become the weights in the model fit below.

7.6.3 The datum: an absolute (T, D) pair

Each scan point records the Sun's true position T at that moment and the *reported drive position* D the mount actually settled on. The scan datum is their difference:

$$\Delta h = h_D - h_T, \qquad \Delta x = \left[(A_D - A_T + 180°) \bmod 360° - 180°\right] \cos h_T. \tag{38}$$

This pair is model-free. Whatever pointing model happened to be loaded during the scan only decided where the raster was aimed, and cancels out of the difference. That is what makes every fit *absolute* rather than a delta composed onto whatever came before, and it removes an entire class of composition error.

It also removes a noise floor. Because D is the position the Due rounded to and actually drove to, rather than the float the scheduler requested, the encoder grid is part of the measurement rather than an unmodelled $0.5°$ uncertainty sitting on top of it; the old quantisation term of $0.144°$ added in quadrature to every scan sigma is gone.

Scan records carry `record_version` (currently 2). Version 1 records were residuals against a model that is no longer knowable, and are skipped by the fit with a log message rather than silently pooled.

7.7 Fitting the model

7.7.1 Design matrix

For N scans the four fitted parameters are $\mathbf{x} = [\mathrm{IE}, \mathrm{IA}, \mathrm{AN}, \mathrm{AE}]^{\mathsf{T}}$ and the design matrix $\mathbf{M}$ is $2N \times 4$: rows $0 \ldots N-1$ are the altitude equations (29) and rows $N \ldots 2N-1$ the azimuth equations (30), i.e.

$$\mathbf{M} = \begin{pmatrix} 1 & 0 & \cos A_i & \sin A_i \\ 0 & 1 & \sin A_i \tan h_i & -\cos A_i \tan h_i \end{pmatrix}, \qquad i = 1 \dots N, \tag{39}$$

stacked altitude block above azimuth block. The observation vector is

$$\mathbf{b} = \left[\, \Delta h_1 - R(h_1),\ \dots,\ \Delta h_N - R(h_N),\ \Delta x_1,\ \dots,\ \Delta x_N \,\right]^{\mathsf{T}}, \tag{40}$$

with refraction subtracted from the altitude half only — refraction is vertical to first order, so the cross-elevation errors are untouched.

7.7.2 Weighted solution

With σ_j the per-scan centroid uncertainty for observation j and $\mathbf{W} = \mathrm{diag}(1/\sigma_j)$, the fit minimises

$$\chi^2 = \left\| \mathbf{W}(\mathbf{M}\mathbf{x} - \mathbf{b}) \right\|^2 \tag{41}$$

by least squares, giving residuals $\mathbf{r} = \mathbf{b} - \mathbf{M}\mathbf{x}$, per-axis RMS residuals, and

$$\chi^2_\nu = \frac{1}{2N - 4} \sum_j \left(\frac{r_j}{\sigma_j} \right)^2. \tag{42}$$

The parameter covariance is

$$\mathbf{C} = \left(\mathbf{M}^{\mathsf{T}} \mathbf{W}^2 \mathbf{M} \right)^{+} \max\left(\chi^2_\nu,\, 1 \right), \tag{43}$$

and the significance of each parameter is $|x_i| / \sqrt{C_{ii}}$. The floor of 1 on the chi-squared scale factor is deliberate: mount quantisation error is partly systematic, so mutually consistent scans give $\chi^2_\nu \ll 1$, and scaling the covariance by it would shrink the parameter errors and overstate the very tilt significance that gates a weak calibration.

7.7.3 Geometry and quality gates

A fit is refused, with a message saying what to collect instead, unless all of the following hold. Azimuth coverage is measured as 360° less the largest gap in the sorted scan azimuths, which is the quantity that actually determines whether $\cos A$ and $\sin A$ are separable.

Table 15: Calibration acceptance criteria

Quantity	Requirement	Why
Valid scans N	≥ 4	Four parameters need two scans in principle; four spread scans are needed to detect poor geometry and give meaningful uncertainties
Azimuth coverage	$\geq 30°$	Below this the two tilt terms are degenerate with the index errors
Condition number	$\leq 10^4$	Direct test that all four parameters are separable
Sun altitude	$0° \leq h_\odot < 89.5°$	$\tan h$ diverges at the zenith
Min. tilt significance	$\geq 3\sigma$	Geometry only says the parameters are separable, not that a tilt was measured
Reduced χ^2	≤ 4	Residuals far above the scan uncertainties mean something other than mount tilt is moving the pointing

The last two are checked again at *apply* time, not just at fit time, and a saved model that predates those checks is rejected rather than pushed.

7.8 Wire format and application

The scheduler builds the document with `pointing_model_document()` and POSTs it to `/pointing/apply`. It deliberately carries the fitted terms and nothing derived from them:

```
{
  "version": 1,
  "fitted_utc": "2026-08-19T13:41:02Z",
  "n_scans": 7,
  "frame": "cross_elevation",
  "terms": {"IE": -0.7213, "IA": 1.9842, "AN": 0.3117, "AE": -0.1042},
  "site": {"lat": 55.902426, "lon": -4.307865},
  "residual_rms_deg": {"alt": 0.06, "xel": 0.08}
}
```

Listing 2: Pointing model document

Unknown term names are ignored, so a richer model can be pushed to older firmware without bricking pointing. A missing `version` or a malformed number rejects the *whole* document and leaves the current model untouched — a half-applied model is worse than none. On success the controller replaces the model under the cross-task lock, writes it to NVS outside the lock, and returns the stored model for confirmation.

`/pointing/clear` zeroes the model and erases the NVS keys. A cleared model and an all-zero model behave identically — both are the identity transform — so it is safe to use after mechanical work without leaving pointing in a special state.

7.9 Operating procedure: a calibration day

1. Ensure the receiver is free: a Sun scan and a calibration day both claim the SDR and will refuse to start if an observation or a manually booted receiver holds it.

2. Start a calibration day from the scheduler (`POST /api/calday/start`) with a scan interval, typically 30 minutes. It waits for sunrise if necessary, runs a scan every interval while the Sun is above 5°, and finishes at sunset.

3. Each hardware scan re-homes the mount first, so every datum is referenced to a freshly measured physical limit. A rejected scan is re-homed and retried once; only the final outcome counts as a failure.

4. Run for a full day, or at least a morning *and* an afternoon set. This is what produces the azimuth spread the tilt terms need; an afternoon-only set will fit and then be refused at the significance gate.

5. Fit with `POST /api/calday/fit`. This writes `pointing_model.json` and a four-panel `calibration_day.png` showing measured against modelled errors in both axes, the residuals, and a summary.

6. Apply with `POST /api/calday/apply`. The gates of Table 15 are enforced here before anything reaches the telescope.

7. Verify: `GET /pointing` on the controller should return the same terms, and they must survive a power cycle.

After mechanical work on the mount, clear the accumulated scan data (`/api/calday/clear`) as well as the model — old scans describe a mount that no longer exists.

8 Observatory host and the controller link

8.1 What the link is

The observatory computer carries a **second Ethernet card**. The controller hangs off it on a private point-to-point link that the host owns completely: it serves the address, resolves names, and routes the link out to the internet. The first card stays on the site LAN and is how people reach the computer.

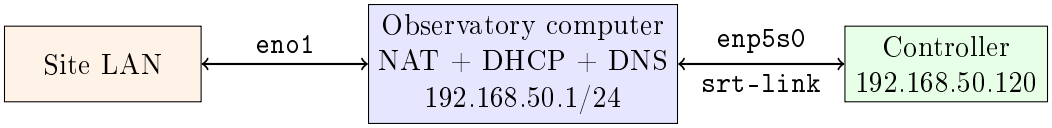

Figure 3: The private controller link. Interface names are illustrative; the card's bus position fixes them.

8.2 Why

On the site LAN the controller had whatever address the site handed it, and that address was hard-coded in seven places in the code and twenty-five in the documentation. Every renumbering was a code change. On a private link the address is permanently ours.

No firmware change is needed for any of this. The controller stays on DHCP and its address is pinned by MAC on the host, so nothing on the controller is configured for this link and the whole arrangement reverses by moving one cable back to the site switch. *That reversibility is the point* and should be preserved by anything built on top of it.

The one thing the link breaks, if built carelessly, is time — and by the argument of Section 5 a clock error is a pointing error, arriving overnight and tedious to diagnose after the fact. Routing the link out is therefore not optional.

8.3 Address plan

The subnet must avoid the site LAN, `192.168.4.0/24` (the controller's own WiFi AP) and `192.168.1.0/24` (common home-router range). `192.168.50.0/24` is used. Nothing depends on the numbers, but changing them means changing `DEFAULT_ETH_*` in the controller's `config.h` too.

Table 16: Address plan

Host	Address
Observatory computer, second card	192.168.50.1
Controller	192.168.50.120 (DHCP, pinned by MAC)

8.4 Hardware

A TP-Link TG-3468: PCIe x1, Realtek RTL8168, driven by the in-kernel `r8169` module, so Ubuntu brings it up with no driver install.

Avoid cheap "Intel I210" listings. A genuine i210-T1 is the better card in the abstract, but real ones are considerably more expensive than the marketplace listings, and those listings are a well-documented counterfeit market. Counterfeits frequently carry cloned or duplicated MAC addresses — exactly the failure this design depends on not happening, since the controller's address is pinned by MAC. The link carries a status poll and a coordinate stream and the controller's PHY is 100BASE-TX anyway: buy on driver reliability, not speed.

8.5 Building the link

Do this while the controller is still on the site switch; nothing is at risk until the cable moves.

```
sudo nmcli connection add type ethernet \
    con-name srt-link \
    ifname enp5s0 \
    ipv4.method shared \
    ipv4.addresses 192.168.50.1/24 \
    ipv4.never-default yes \
    ipv6.method ignore
```

Listing 3: Create the shared connection

`ipv4.method shared`
: The important one. It sets the address, enables IP forwarding, installs the NAT rules, and runs a dnsmasq on the interface providing both DHCP and a DNS forwarder — all four in one setting.

`ipv4.never-default yes`
: Stops the host routing general traffic down a link with only a microcontroller on the far end.

`ipv6.method ignore`
: Nothing here speaks IPv6.

The controller's address is then pinned by MAC so it can stay on DHCP:

```
dhcp-host=70:4B:CA:58:59:8B,192.168.50.120,srt-controller
```

Listing 4: /etc/NetworkManager/dnsmasq-shared.d/srt.conf

Shared mode leases from `.10` upward and a `dhcp-host` entry takes precedence within that range, so `.120` is safe. Activation without a cable attached succeeds; the interface stays `NO-CARRIER` until the controller is moved across, which is normal. Verify with `sysctl net.ipv4.ip_forward` (must be 1), `nft list ruleset | grep -i masquerade`, and `pgrep -af "dnsmasq.*enp5s0"`.

8.6 Firewall

Note first that `systemctl is-active ufw` reports "active" even when the firewall is not enforcing. `sudo ufw status` is the real answer.

```
sudo ufw limit 22/tcp                          comment 'ssh (rate limited)'
sudo ufw allow in on enp5s0                    comment 'SRT private link'
sudo ufw route allow in on enp5s0 out on eno1 comment 'SRT link NAT out'

sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw default deny routed
sudo ufw enable
```

Listing 5: Firewall rules

Two of those are non-obvious and both were learned the hard way.

- `route allow ...out on eno1` **is what keeps the controller's clock working.** The default forward policy is deny, which silently blocks the NAT. The controller then never syncs time again and the mount mispoints overnight.

- **`allow in on enp5s0` is what keeps OTA firmware updates working.** `espota` sends its invitation to the controller and then expects the controller to connect *back* to the host on an ephemeral port — a different one every run — so it cannot be allowed by port number. Without this rule the upload fails with the unhelpful `No response from device`, because the invitation succeeds and only the callback is blocked. Trusting the link interface as a whole is reasonable: it carries exactly one device, on a subnet this host owns and serves.

Nothing needs an inbound rule on the site interface. In particular do not open port 5000 — see Section 2.

8.7 Verification

```
ping -c3 192.168.50.120
curl http://192.168.50.120/ping          # -> ok
curl http://192.168.50.120/network       # eth_ip and the DNS diagnostics
curl http://192.168.50.120/status        # live drive and true alt/az
getent hosts srt-controller.local        # mDNS, needs avahi on the host
```

Listing 6: Basic checks from the observatory computer

Then the check that is easy to get wrong. **`/time/status` reporting `"source":"NTP"` does not prove the link routes.** The firmware carries a numeric NTP fallback behind the pool name, so the clock syncs whether or not DNS works, and will report `NTP` on a host that cannot resolve anything at all — a true statement about the clock and a useless one about the network.

What proves it is a *fresh sync after a reboot*. Reboot the controller (an OTA upload will do), then query `/time/status`: `sync_count` must have reset to 1, confirming a real reboot rather than a stale reading, with a small `last_sync_age_s`. That combination means SNTP packets left the host, reached a time server and came back.

DNS itself can be confirmed either from `/network` (Section 5) or by temporarily adding `log-queries` to the dnsmasq drop-in and watching for `query[A] pool.ntp.org from 192.168.50.120` in the NetworkManager journal.

Finish by running one OTA upload deliberately, since that is the operation most dependent on the firewall being right:

```
cd esp32_controller_arduino
~/.platformio/penv/bin/pio run -e wt32-eth01-ota -t upload
```

8.8 Recovery

The controller has independent safety nets, so a wrong address is not a bricking risk:

1. Move the cable back to the site switch. Because the controller was never reconfigured, this restores the previous state exactly.
2. The WiFi AP stays active — SSID `SRT_Controller`, UI at `http://192.168.4.1` — regardless of the Ethernet settings.
3. Invalid static configuration falls back to DHCP, so a malformed address leaves the controller reachable rather than dark.
4. Last resort is the FT232-and-buttons serial flash at the telescope.

8.9 What the link deliberately does not do

Access to the controller from other machines on the site network is *not* set up. The host NATs outbound for the controller; nothing routes inbound. Providing it would need a forwarding rule or reverse proxy, which partly re-exposes the thing the private link exists to protect. Remote use at Acre Road is `waypipe` into the observatory computer instead.

The site interface keeps whatever the site gives it. Nothing here depends on its address: the masquerade rule matches on the link subnet, the firewall rules name interfaces rather than addresses, and `never-default` keeps the link out of the routing decision.

The full build procedure, with the commands in order and the checks between them, is in `docs/OBSERVATORY_HOST_SETUP.md`.

9 Beam, radiometry and velocity frames

Everything so far has been about putting the beam somewhere. This section is about the beam itself, about what the numbers coming out of the receiver mean, and about the one correction the system deliberately does *not* apply.

9.1 Beam and aperture

The dish is 3.0 m across and the observing wavelength is

$$\lambda = \frac{c}{\nu_{\mathrm{H\,I}}} = \frac{2.998 \times 10^{8}}{1.4204 \times 10^{9}} = 0.2111 \text{ m}. \tag{44}$$

For a uniformly illuminated circular aperture the half-power beamwidth is

$$\theta_{\mathrm{FWHM}} \simeq 1.22\,\frac{\lambda}{D} = 1.22 \times \frac{0.2111}{3.0} = 0.0858 \text{ rad} = 4.9^{\circ}, \tag{45}$$

and a realistic illumination taper widens this slightly. This single number sets the scale of nearly every tolerance in the preceding sections:

- The 0.5° encoder quantum is about one tenth of a beamwidth, so pointing quantisation costs almost nothing in signal.
- The residual pointing error after calibration, a few hundredths of a degree, is negligible against it — but the *uncalibrated* error of about 2° is nearly half a beamwidth, which is why the calibration of Section 7 is worth the trouble.
- The Sun-scan raster spacing of 1.5° is roughly 0.3 beamwidths, and a 5×5 grid spans ±3°, comfortably over the half-power points in both axes — which is what the Gaussian fit needs to find a centroid.

The corresponding beam solid angle for a Gaussian beam is $\Omega_{\mathrm{b}} \simeq 1.133\,\theta^{2}_{\mathrm{FWHM}} \approx 27$ square degrees. An unresolved source of solid angle Ω_{s} and brightness temperature T_{b} is therefore diluted to an antenna temperature

$$T_{\mathrm{A}} \simeq \eta_{\mathrm{mb}}\, T_{\mathrm{b}}\, \frac{\Omega_{\mathrm{s}}}{\Omega_{\mathrm{b}}}, \tag{46}$$

with $\eta_{\mathrm{mb}} \approx 0.7$ a typical main-beam efficiency for a dish of this kind. Equation (46) is why the Sun is usable as a pointing calibrator: its 0.53° disc is only $\Omega_{\mathrm{s}}/\Omega_{\mathrm{b}} \approx 0.008$ of the beam, but its 21 cm brightness temperature is of order 10^{5} K, so it still arrives as an antenna temperature of order 10^{3} K — far above everything else in the sky and unmistakable in a raster. That is the figure the receiver's on-Sun power-bar ceiling (`CAL_BAR_SUN_K`) is set from.

9.2 System temperature

The receiver's default blank-sky system temperature is 70 K, built up as follows.

Table 17: Blank-sky system temperature budget (`DEFAULT_CAL_TEMP_K`)

Contribution	Temperature	Source
Low-noise amplifier	43 K	SAWbird+ H1, noise figure ≈ 0.6 dB
Sky	10 K	CMB, atmosphere, galactic background
Spillover and feed losses	17 K	Ground pickup past the dish edge
Total	**70 K**	

The amplifier figure follows from the noise figure in the usual way,

$$T_{\mathrm{rx}} = T_0 \left(10^{F/10} - 1\right) = 290 \left(10^{0.06} - 1\right) = 43 \text{ K}, \tag{47}$$

with $T_0 = 290$ K the reference temperature. The remaining two terms are estimates rather than measurements; a measured T_{sys} is what the calibration below is for.

9.3 The radiometer equation

The RMS fluctuation in a single spectral channel of width $\Delta\nu$ after an integration τ is

$$\Delta T = \frac{T_{\mathrm{sys}} + T_{\mathrm{A}}}{\sqrt{n_{\mathrm{pol}}\,\Delta\nu\,\tau}}, \tag{48}$$

where n_{pol} is the number of polarisations summed (one, for this receiver). The signal appears in the numerator as well as the system temperature because the source is itself noise: a channel sitting on a bright H I line fluctuates more than a channel beside it, and the simulator of Section 11 models exactly this.

As a worked example, at the default settings — 2.4 MHz of bandwidth over 4096 channels, so $\Delta\nu = 586$ Hz — and taking $T_{\mathrm{sys}} = 200$ K on a modest line,

$$\Delta T = \frac{200}{\sqrt{586 \times 60}} = 1.1 \text{ K per channel in one minute.} \tag{49}$$

Galactic H I at low latitude reaches tens of kelvin, so the line is visible in a single minute; a high-latitude profile of a few kelvin needs tens of minutes, and Equation (48) is how that observation is planned.

9.4 Temperature calibration

Two calibration paths exist, and only one of them is currently automated.

9.4.1 Single-point blank-sky calibration

The receiver GUI offers a one-click calibration: with the dish on blank sky, the mean total power over the sliding window is declared to be the entered temperature, giving a scale factor

$$k = \frac{T_{\mathrm{cal}}}{\bar{P}} \quad [\text{K per arbitrary unit}], \tag{50}$$

after which the total-power chart reads in kelvin and the power bar is re-spanned from just below blank sky (50 K) to the approximate on-Sun level. The scale is invalidated automatically if the bandwidth or FFT size changes, because those alter the raw power scale. It is a one-point calibration against an *assumed* T_{sys}, so it fixes the units, not the accuracy. Section 17.8.3 shows how a blank-sky and Sun pair measures the number that is currently typed into that box.

9.4.2 Noise diode and the Y-factor

The calibrator is a noise diode on GPIO 26 of the Due, switchable from the web UI, the API and the schedule, and its state is recorded both in the HDF5 `calibrator` attribute and in a `_cal` filename suffix. At present the system *records* the diode state and does not derive a gain solution from it. The intended use, when a diode of known excess temperature $T_{\rm cal}$ is characterised, is the standard Y-factor:

$$Y = \frac{P_{\rm on}}{P_{\rm off}}, \qquad T_{\rm sys} = \frac{T_{\rm cal}}{Y-1}, \qquad T_{\rm A} = T_{\rm sys}\left(\frac{P_{\rm source}}{P_{\rm off}} - 1\right). \tag{51}$$

Because the diode fires ahead of the low-noise amplifier, the same measurement tracks gain drift through the observation, which the blank-sky method cannot. Interleaving on and off spectra is therefore the upgrade path for absolute work; the metadata needed to do it offline is already being written to every file.

9.5 Velocity frames — and the LSR correction the system does not apply

The receiver's spectrum carries a second axis in velocity, computed in the radio convention

$$v = c\,\frac{\nu_0 - \nu}{\nu_0}, \qquad \nu_0 = 1420.405752\ \text{MHz}, \tag{52}$$

equivalently $1\ \mathrm{km\,s^{-1}} \equiv 4.74\,\mathrm{kHz}$ at this frequency.

That axis is topocentric. No correction to the Local Standard of Rest is applied, and this is deliberate rather than an oversight: the correction depends on the pointing direction, and the receiver process does not know where the dish is looking. The consequence must be understood by anyone using the data, because it is not small:

Table 18: Velocity terms between the topocentric and LSR frames

Term	Maximum	Period
Earth rotation	$0.46\,\mathrm{km\,s^{-1}}$	Daily
Earth orbital motion	$29.8\,\mathrm{km\,s^{-1}}$	Annual
Solar motion towards the LSR apex	$20.0\,\mathrm{km\,s^{-1}}$	Fixed per direction
Combined	$\sim 50\,\mathrm{km\,s^{-1}}$	$\equiv 237\,\mathrm{kHz}$

A quarter of a megahertz is a large fraction of a galactic H I profile, and the annual term reverses over six months, so uncorrected spectra of the same source taken in different seasons will not superpose.

The correction is applied in analysis. Writing $v_{\rm topo}$ for the axis as recorded, $v_{\rm bary}$ for the observer's barycentric radial velocity towards the source at the time of observation, and $\hat{n}$ for the unit vector to the source,

$$v_{\rm LSR} = v_{\rm topo} + v_{\rm bary} + \mathbf{U}\cdot\hat{n}, \tag{53}$$

where $\mathbf{U}$ is the kinematic solar motion, which defines LSRK as $20.0\,\mathrm{km\,s^{-1}}$ towards RA $18^{\rm h}$, Dec $+30°$ (B1900). Every quantity needed is in the HDF5 file: the UTC timestamps, the target coordinates and the observer position. In practice $v_{\rm bary}$ comes from `astropy`'s `SkyCoord.radial_velocity_correction("barycentric", ...)`, and the apex projection is a single dot product. The reference implementation is `frame_offset()` in `astro_simulator/astro_simulator.py`, which converts between the LSR, barycentric and topocentric frames in exactly this way; anything written to correct the observed data should agree with it.

10 H1 receiver and observation scheduler

10.1 Receiver overview

`b210_h1_receiver.py` is a GNU Radio spectrum analyser for 21 cm observations, offering real-time FFT with configurable integration, a live spectrum and waterfall display (PyQtGraph), and continuous logging to HDF5.

Table 19: Supported SDR platforms

SDR	Frequency range	Sample rate	Notes
Ettus B210	70 MHz – 6 GHz	up to 56 MHz	Recommended
RTL-SDR (R820T)	24 – 1766 MHz	up to 2.4 MHz	Budget option
Demo	—	—	Simulated data, no hardware

```
python b210_h1_receiver.py --sdr b210 --gain 40 --sample-rate 2.4e6
python b210_h1_receiver.py --sdr rtlsdr --gain 45
python b210_h1_receiver.py --sdr demo
```

Listing 7: Receiver command line

Two receiver instances conflict over the default `h1_data.h5` file lock, which is the usual cause of a receiver that exits immediately.

10.2 HDF5 output format

Table 20: HDF5 dataset structure

Dataset	Shape	Description
`frequency_hz`	$(N_{\text{fft}},)$	Frequency axis in Hz
`spectra_linear`	$(N_{\text{spectra}}, N_{\text{fft}})$	Power spectra, linear units
`timestamps`	$(N_{\text{spectra}},)$	Unix timestamps
`integration_times`	$(N_{\text{spectra}},)$	Actual integration per spectrum

Spectra are stored in linear power, not dB, to preserve radiometric accuracy for calibration. Each file has a fixed frequency axis and FFT width; if the frequency, sample rate or FFT size changes during operation the current file is closed and a new timestamped segment started, so the datasets remain internally consistent.

Table 21: HDF5 root attributes (always present)

Attribute	Description
`sdr_type`	`b210`, `rtlsdr` or `demo`
`center_freq_hz`	Centre frequency in Hz
`sample_rate_hz`	Sample rate in Hz
`fft_size`	Number of FFT bins
`gain_db`	RF gain in dB
`nominal_integration_time`	Target integration time in seconds
`created`	ISO 8601 UTC file creation timestamp

Table 22: HDF5 observation metadata (added when launched by the scheduler)

Attribute	Description
`obs_name`	Observation name from the schedule
`coord_system`	`altaz`, `radec`, `galactic`, `object`, `satellite`
`object_name`	Solar-system object if applicable
`coord1_deg/min/sec`	Target coordinate 1 (Alt, RA or l)
`coord2_deg/min/sec`	Target coordinate 2 (Az, Dec or b)
`calibrator`	1 if the noise source was on
`duration_minutes`	Scheduled duration
`start_date`, `start_time`	Scheduled start
`tle_text`	TLE data for satellite observations

`read_h1_data.ipynb` reads these files, shows the metadata and plots waterfalls and mean spectra. All averaging is done in linear power; dB conversion is for display only.

10.3 Observation scheduler

`h1_web_scheduler.py` is a Flask application serving a tabbed interface on `http://localhost:5000`. Configuration persists in `scheduler_config.json` and activity is written to a rotating `scheduler.log`.

10.3.1 Interface

Scheduler tab
Add, edit, clone and delete observations, with clash prevention, late-start recovery, preemption, a countdown to the next slot, audio notification, JSON import/export and a "Clear Past" action. Running observations are protected from editing.

Configuration tab
Controller URL and fallbacks, observer location, slew and homing timeouts, Python and PlatformIO paths, receiver interpreter, data folder, minimum elevation, banner text and sound.

Calibration tab
Single Sun scan, calibration day, model fit, plot and apply — the workflow of Section 7.9.

Log tab
Last N lines of `scheduler.log` with auto-refresh. The file captures DEBUG including periodic schedule dumps; the console is filtered to INFO.

10.3.2 Runtime behaviour

- The scheduler re-executes itself under `receiver_python_path` (radioconda by default) unless it is already running there, so receiver subprocesses always have GNU Radio available.
- It is launched from the desktop **Start SRT Software** item via the VS Code workspace `folderOpen` task, which runs `start_scheduler.sh`. That wrapper reuses an already-serving instance rather than starting a second one that would die on the bound port. It is deliberately *not* a systemd service: a unit would contend for port 5000 with the task, and the scheduler should not come up unattended.

- A manual receiver boot is for warm-up and testing only. Scheduled observations stop it before claiming the SDR, and `/api/receiver/status` reports whether the running process is manual or observation-owned.
- A scheduled observation always wins: it cancels a running Sun scan or calibration day and waits up to `SUN_SCAN_PREEMPT_TIMEOUT` (600 s) for the SDR before starting.
- `start_observation` claims the start briefly under `process_lock`, then does the pointing and slew-wait *unlocked* and abortable through `start_abort`. Blocking work must not be reintroduced under that lock.
- Failed or short-lived starts back off after `MAX_START_FAILURES` (3) attempts per schedule slot.
- CORS is restricted to the configured controller origins, so the scheduler buttons embedded in the controller's own page work only if `srt_controller_url` and its fallbacks match the origin that page is served from.
- `POST /api/config` accepts only keys already present in the default configuration, and observation filenames are constrained to stay inside the data folder.

10.3.3 Coordinate systems and drift scans

Table 23: Supported observation coordinate systems

System	Description	Tracking
Alt/Az	Direct horizontal pointing	Fixed position
RA/Dec (J2000)	Equatorial	Sidereal tracking
Galactic (l, b)	Galactic	Sidereal tracking
Solar-system object	Sun or Moon	Ephemeris tracking
Satellite (TLE)	Two-line element set	SGP4, 1 Hz updates
Drift	Fixed alt/az, source transits the beam	None, by design

A drift scan parks the dish at the alt/az a source will occupy at a chosen beam-crossing time T and lets the sky carry the source through the beam. `GET /api/drift_preview` returns the resulting pointing, reachability warnings against the horizon and the azimuth dead zone, and the source's next meridian transit — the classical choice of T, because the source moves slowest in altitude there and the crossing is symmetric.

For satellites, PyEphem propagates the TLE with SGP4 [13] and the scheduler sends `/direct` updates every second. TLEs can be pasted, loaded from file, or fetched from CelesTrak by name or NORAD number.

10.3.4 Observation parameters

Table 24: Observation schedule parameters

Parameter	Description
Name	Descriptive name
Start date/time	Local time; end time computed from the duration
Duration	Minutes
Coordinates	Target position in one of the systems above
Centre frequency	Default 1420.405 MHz
Bandwidth	Sample rate in MHz
Gain	RF gain in dB
Channels	FFT size
Integration time	Seconds per averaged spectrum
SDR type	B210, RTL-SDR or demo
Calibrator	Noise source on/off (`_cal` filename suffix when on)
When done	Stay, Home (run the physical homing sequence), or Stow (Alt 90, Az 180)

Note that "Home" here runs the Due's homing sequence — a mechanical re-reference — while "Stow" parks at the drive position of Section 5. They are different operations and take different lengths of time.

10.3.5 Automated workflow

1. The user creates an observation in the web interface.
2. At the scheduled time the scheduler sends the pointing command to the controller.
3. It polls `/status` and waits for `is_slewing` to clear, comparing drive against drive.
4. The calibrator noise source is set as configured.
5. For satellites, a background thread begins 1 Hz position updates.
6. The receiver is launched under the radioconda interpreter.
7. Data is written to HDF5 in linear power with the observation metadata attached.
8. On completion the calibrator is turned off and the "when done" action is carried out.

10.3.6 Tests

The scheduler and calibration code are covered by `pytest`:

```
python -m pytest receiver_scheduler/test_scheduler.py receiver_scheduler/
    test_sun_scan.py
```

One known false failure: `TestFlaskAPI::test_post_config` fails on the observatory host because it makes a real HTTP call, reaches the live controller, and the controller then syncs the observer location back over the value the test just set. It is a test-isolation problem, unrelated to whatever is being changed; check it against a clean tree before chasing it.

11 Sky simulator

`astro_simulator/` predicts what this telescope should see. It is not part of the observing chain, but it is how an observation is planned and how a puzzling spectrum is checked against expectation — and it can hand its pointing to the real telescope.

11.1 Sky models

Two all-sky datasets back the simulator.

H I line
: The HI4PI survey [6], with a 16.2′ beam and 1.29 km s^{-1} channels — far finer than the 4.9° beam of Section 9.1, so the simulated spectrum is a genuine beam-weighted average of the survey brightness temperature rather than an interpolation.

1420 MHz continuum
: The Stockert/Villa-Elisa survey [7], on the same grid, with the zero level (CMB plus isotropic, ≈ 3.2 K) stored separately and the strong sources — Cygnus A, Cassiopeia A, Taurus A — removed for analytic re-insertion, because they are saturated or blanked in the survey itself.

The full HI4PI cube is 33 GiB. The repository ships a compact form of about 23 MB, block-averaged to 0.5° pixels, smoothed to 1° total resolution, trimmed to $|v| \le 470$ km s^{-1}, thresholded at 3σ, quantised to 0.01 K in `int16` and LZMA-compressed. It is exact to better than 0.5% for beams of 1.6° and above, which covers every dish this simulator is meant for; the full cube is fetched from CDS only if a request falls outside what the compact data can honour.

11.2 Instrument model

The simulator computes the natural beam from the aperture exactly as in Section 9.1, convolves the survey to it, adds the continuum offset, and applies radiometer noise using Equation (48) — including the signal term, so channels on a bright line are correctly noisier than their neighbours.

Table 25: Simulator parameters and defaults

Option	Default	Unit	Meaning
`--dish`	3.0	m	Aperture; sets the beam as $1.22\lambda/D$
`--eta`	0.7	—	Main-beam efficiency
`--bw`	2.0	MHz	Bandwidth
`--nchan`	native	—	Spectrometer channels
`--tsys`	200	K	System temperature
`--tint`	60	s	Integration time
`--npol`	1	—	Polarisations summed
`--controller`	`http://192.168.50.120`	—	Target for *Realise*

11.3 Velocity frames

The velocity axis can be expressed in the LSR, barycentric or topocentric frame, and `frame_offset()` is the reference implementation of the transformation discussed in Section 9.5. Since the receiver records topocentric velocities and the simulator works natively in LSR, comparing a real spectrum with a simulated one *requires* the correction — which makes the simulator a practical check that it has been applied correctly.

11.4 Modes

The lower panel is modal, following the sky map selected above it.

Spectrum
With the H I map, clicking anywhere on the sky gives the dish spectrum for that direction, with editable pointing, beam, T_{sys}, integration time, bandwidth, band centre, channel count and velocity frame. A target list covers the standard H I pointings plus the continuum sources and the Sun and Moon; site horizon and visibility overlays and a live RA/Dec and alt/az readout show whether the direction is reachable from Glasgow at all.

Drift scan
With the continuum map, the panel shows band-averaged T_{A} for the sky drifting through a beam parked on the current target — centred on beam-centre transit, over the duration set in the scan box, with per-sample noise. This is the planning tool for the drift observations of Section 10.

11.5 Realise

The *Realise* button hands the simulation to the telescope. In spectrum mode it starts galactic tracking of the current pointing (`/track/galactic`); in drift mode it cancels tracking and parks the dish at the drift-scan start position (`/direct`), so that the target transits the beam centre half a scan later. It refuses if the resulting position is below the horizon.

This is the only path by which the simulator commands hardware, and it uses the ordinary controller API described in Appendix C — there is no private channel.

12 Installation

12.1 Prerequisites

- PlatformIO (VS Code extension or CLI)
- Python 3 with `esptool`
- Radioconda for the receiver and scheduler

PlatformIO is invoked by absolute path, since `pio` is not on `PATH`:

Platform	Binary
Windows	`~/.platformio/penv/Scripts/pio.exe`
Observatory Linux host	`~/.platformio/penv/bin/pio`

12.2 Arduino Due drive firmware

```
~/.platformio/penv/Scripts/pio.exe run -e due            # build
~/.platformio/penv/Scripts/pio.exe run -e due -t upload  # flash
~/.platformio/penv/Scripts/pio.exe run -e simulation     # host-side simulation
~/.platformio/penv/Scripts/pio.exe device monitor -b 115200
```

12.3 WT32-ETH01 controller firmware

```
cd esp32_controller_arduino
~/.platformio/penv/Scripts/pio.exe run                          # wt32-eth01 (default)
~/.platformio/penv/Scripts/pio.exe run -e wt32-eth01 -t upload   # first, serial flash
~/.platformio/penv/Scripts/pio.exe run -e wt32-eth01-ota -t upload  # routine, over Ethernet
~/.platformio/penv/Scripts/pio.exe test -e native               # host unit tests
```

The WT32-ETH01 is the deployed board and the default build; it must always compile clean. The `esp32s3` environment is legacy — that board is no longer used — and is kept building only when convenient.

12.4 Configuration

The observer location defaults live in the controller's `config.h` and are the *true* site position:

```
#define OBSERVER_LAT 55.902426  // degrees north
#define OBSERVER_LON -4.307865  // degrees east (west negative)
```

They can be changed at runtime from the Settings tab and are persisted to NVS. The same numbers appear in the scheduler's default configuration and in `sun_scan.py`, and all three must agree. Nothing should ever write a *fictitious* observer position to compensate for pointing error; that is what the pointing model is for.

12.5 Receiver prerequisites

```
# 1. Install radioconda: https://github.com/ryanvolz/radioconda/releases
# 2. Activate it
conda activate radioconda
# 3. Install the remaining Python dependencies
cd receiver_scheduler
pip install -r requirements.txt
```

Radioconda supplies GNU Radio, PyQt5, NumPy, SciPy and h5py; `requirements.txt` adds Flask, PyEphem and pytest.

13 Testing and verification

There are four ways to exercise this system without a telescope, and between them they cover almost everything except the motors themselves. Anyone changing the code should know which one applies before starting.

Table 26: Test paths

Path	Covers	Command
Drive simulation	Due firmware	`pio run -e simulation`
Native unit tests	Coordinate and pointing maths	`pio test -e native`
Pytest	Scheduler, Sun scan, model fit	`pytest receiver_scheduler/`
Due emulator	Controller firmware on the bench	`python tools/due_emulator.py -port COM7`

13.1 Drive firmware in simulation

Described in Section 4: GPIO, PWM, ADC and interrupts are replaced by macros and `simulatePulses()` advances position from the commanded PWM. Both the `due` and `simulation` environments must build clean, and any change to the homing flow or to in-loop motor control must still complete in simulation — the homing while-loops call `simulatePulses()` themselves precisely so that they can.

13.2 Native unit tests

The `native` environment compiles `coordinates.cpp` and `pointing.cpp` on the host and runs `test/test_coordinates` and `test/test_pointing` against them — around 110 assertions covering Julian dates, sidereal time, precession, the horizontal and galactic transforms, the ephemerides, the pointing model terms, and the round-trip identity $\texttt{driveToTrue}(\texttt{trueToDrive}(x)) = x$.

The important property is that these are the *flashed* sources, not copies: `test/shim/` supplies only the small parts of `Arduino.h` and `Preferences.h` that `pointing.cpp` touches. A test that passes against a transcribed copy of an algorithm proves nothing about the firmware.

13.3 Scheduler and calibration tests

```
python -m pytest receiver_scheduler/test_scheduler.py receiver_scheduler/
    test_sun_scan.py
```

Roughly 156 tests covering scheduling logic, clash and late-start handling, the sun-scan geometry, the design matrix — including a check of the pointing-model matrix against numerically rotated horizon coordinates — the fit, the quality gates and the wire format. The known false failure of `TestFlaskAPI::test_post_config` on the observatory host is described in Section 10.

13.4 The Due emulator

`tools/due_emulator.py` presents the Due end of the UART protocol over a 3.3 V USB-to-UART adapter, so the controller firmware can be exercised on the bench with no telescope attached. It reproduces the 1 Hz status stream in the exact positional format, the full command set, quantised motion at 0.5°, homing, and plausible motor currents.

> **The ESP32 is not 5 V tolerant.** Set the adapter jumper to 3.3 V, wire TX and RX crossed to IO4 and IO14 with a common ground, and do not use the board's TX0/RX0 pins — those are the programming console.

Its value is in the failures it can inject, which are exactly the ones that are hard to produce deliberately on a working telescope:

Table 27: Emulator fault-injection commands

Command	Effect
`fault <text>`	Enter FAULT with the given message
`clear`	Clear the fault, as a power cycle would
`pos <alt> <az>`	Teleport the reported position
`quiet on|off`	Stop or resume the status stream, testing the timeout UI
`garbage`	Send one malformed line, testing the parser
`split`	Send the next status line in two delayed chunks

`split` deserves particular note: line splicing on the UART is what produced the corrupt status readings that led to the rate limit of Section 4, and it is otherwise reproducible only by overloading a real link. The `--capture` option logs every raw chunk with timestamps, which is how interleaved command lines from the controller are detected.

The issues labelled `needs-hardware` cannot be closed on a development host and need either this emulator, a bench ESP32, or the mount itself.

14 Operation

14.1 Startup sequence

1. Power on the Arduino Due and the controller.
2. The Due zeroes its current sensors, then runs the three-phase homing sequence of Section 4.3, finishing at the configured home position and reporting `Ready`. Allow 30–60 s.
3. The controller brings up Ethernet, obtains its pinned DHCP address, starts the WiFi AP, loads the settings and the stored pointing model from NVS, and begins SNTP.
4. Confirm on the web interface that the clock has synced and that the pointing model is loaded (`pointing_loaded` in `/status`).

A controller that comes up with no pointing model is not broken — it applies the identity transform plus refraction — but it will point about two degrees off, which is most of a beam width.

14.2 Reaching the web interface

Normally, from the observatory computer, either `http://192.168.50.120/` or the mDNS name `http://srt-controller.local/`.

If that fails, connect to the WiFi network `SRT_Controller` with password `radio1420`, and browse to `http://192.168.4.1`. The AP is up regardless of the Ethernet configuration and is the standing way back in.

14.3 Observing

1. Enter the target: RA (0–24 h) and Dec (−90 to +90°); galactic l (0–360°) and b; or alt/az.
2. “Track” for continuous tracking, “Go To” for a single slew.
3. One-click targets are provided for the Sun, the Moon, the galactic plane (Section 5.6.3) and stow.
4. Watch the status readout: it shows the drive position, the true sky position it corresponds to, the derived RA/Dec and l, b, and whether the mount is slewing.

Targets below the observing horizon are refused outright; a tracked target that sets parks the dish at stow and resumes automatically on rising.

14.4 Safety features

- **Two-tier position limits** — software limits inside the physical stops, with a bounded transient tolerance
- **Current limiting** on the filtered current, with continuously re-tracked zero offsets

- **Stall detection** requiring both a settled drive start and an absence of pulses
- **Fault-flag persistence** filtering, so switching transients do not trip a fault
- **Pointing-correction clamps**, so a broken model cannot fling the mount at a limit switch
- **FAULT latching**, requiring an explicit reset

15 Troubleshooting

15.1 System does not start

Symptom	Check
No serial output	USB cable, correct port
Stuck in homing	Motor and encoder connections
Immediate fault	Driver power, fault-flag wiring
Homing aborts "not responding"	Encoder wiring, or `DEBOUNCE` set too long

15.2 Motors do not move

Symptom	Check
PWM output but no motion	Driver reset pins (should be HIGH)
No PWM output	PWM pins 8 and 10
Motors run backwards	Direction wiring, and the `*_DIR_INVERT` macros

15.3 Position errors

Symptom	Check
Wrong position after homing	`HOMEALT`/`HOMEAZ`
Position drifts	Encoder debounce; increase `DEBOUNCE`
Total pulse loss at speed	Debounce window comparable to the pulse interval
Overshoots target	Increase `RAMPDOWN`
Jerky motion	Increase `RAMPUP`

15.4 Pointing errors

Symptom	Check
Consistent $\sim 2^\circ$ offset	`pointing_loaded` false — no model applied
Offset returned after a reboot	Model applied to the offset boxes, not `/pointing/apply`
Error grows through the day	Clock; check `sync_count` and `last_sync_age_s`
Fit refused, low significance	Scans span too little azimuth; add a morning *and* afternoon set
Fit refused, high χ^2_ν	Something other than tilt is moving the pointing — check for mechanical slip
Altitude error doubles at low elevation	Refraction counted twice: the two `refraction` implementations have diverged
Dish parks at drive azimuth 170	Stow being routed through the pointing model

15.5 Network and web interface

Symptom	Check
Cannot reach 192.168.50.120	`srt-link` up, cable on the second card, DHCP lease in the journal
Reachable but the page has no data	Due serial link and baud rate
Scheduler buttons dead on the controller page	CORS: `srt_controller_url` must match the page's origin
OTA "No response from device"	Missing `ufw allow in on enp5s0` for the ephemeral callback
Clock never syncs	Missing `ufw route allow` rule; NAT is being dropped
`dns_after_wifi` is `0.0.0.0`	Expected; lwIP clears the resolver list when the AP starts
`malformed_status` climbing	Serial1 status flooding; check the rate limit in `outputStatus()`

15.6 Coordinates and Stellarium

Symptom	Check
Position does not match the sky	Observer latitude and longitude, and time sync
Errors above 1°	Time sync status first, pointing model second
Sun/Moon out by ∼20 arcmin	Normal for the simplified lunar ephemeris
Stellarium will not connect	Address and port 10001
Connects but will not slew	Select the object, then Ctrl+1
Stellarium marker offset from the dish	`driveToTrue()` not being applied to the reported position

16 Outstanding work and known limitations

This document describes a working instrument, not a finished one. Two categories of open item are worth stating plainly so that the preceding sections are not read as a claim of completeness.

16.1 Known limitations

No LSR correction in the recorded data
Section 9.5. The velocity axis is topocentric and the correction must be applied in analysis. This is the limitation most likely to produce a wrong scientific result if overlooked.

No absolute temperature calibration
Section 9. The noise diode is switched and recorded but not used for a gain solution, and the blank-sky calibration is a single point against an assumed T_{sys}.

Pointing model fitted from the Sun alone
Only IE, IA, AN and AE are constrained; CA, NPAE and TF exist in the transform and the wire format but are left at zero. A source list spanning a wider range of elevations would constrain the rest.

Simplified lunar ephemeris
5–10 arcminutes, which is negligible against a 4.9° beam but would not be for a larger dish.

No inbound route to the controller from the site LAN
Section 8, deliberately.

16.2 Where the open work is recorded

Outstanding work lives in GitHub issues on the repository, not in this document and not in `TODO` files — two earlier review documents were folded into issues and removed, and their content remains in the git history. Each issue carries the finding, `file:line` references, a suggested approach and its verification steps.

Issues labelled `needs-hardware` cannot be verified on a development host: they need a bench ESP32, the Due emulator of Section 13, or the mount itself, and elsewhere are compile-check-only. The most significant open item at the time of writing is the missing cross-task locking in parts of the controller firmware.

Several issues record conclusions that took substantial analysis and are easy to re-derive incorrectly — in particular, that no tracking lead should be added because rounding already performs it, and that scans must record absolute (T, D) pairs rather than composing deltas (Section 7). The reasoning is stated in each issue; re-deriving from first principles tends to reach the plausible-but-wrong version.

17 The 21 cm line: expected signals and the temperature scale

Section 9 established the beam and the units. This section is about the astronomy those units are for: what the 21 cm line is, what antenna temperature this particular telescope should see from it, what system temperature to expect, and how to measure that system temperature using nothing but the sky and the Sun.

17.1 The hyperfine transition

Neutral atomic hydrogen in its ground state has two hyperfine levels, according to whether the proton and electron spins are parallel ($F = 1$) or antiparallel ($F = 0$). The splitting corresponds to

$$\nu_0 = 1420.405751768 \text{ MHz}, \qquad \lambda_0 = 21.106 \text{ cm}. \tag{54}$$

The transition is magnetic-dipole and strongly forbidden [8]: the Einstein coefficient is

$$A_{10} = 2.869 \times 10^{-15} \text{ s}^{-1}, \tag{55}$$

so a single atom radiates about once every 11 million years. It is observable at all only because the Galaxy contains of order $10^{10}\, M_\odot$ of neutral hydrogen, and because collisions keep the two levels close to thermal equilibrium at a *spin temperature* T_s, which in the cold neutral medium tracks the kinetic temperature.

The level populations follow $n_1/n_0 = 3\exp(-h\nu_0/kT_\mathrm{s})$, and since $h\nu_0/k = 0.068\,\mathrm{K}$ is far below any astrophysical temperature, the ratio is 3 to within a part in a thousand: three quarters of all H I atoms are in the upper state at any time, regardless of temperature. This is why 21 cm emission measures *how much* hydrogen there is, almost independently of how hot it is.

17.2 What the line measures

The observed brightness temperature along a line of sight is

$$T_\mathrm{B} = T_\mathrm{s}\left(1 - e^{-\tau_\nu}\right), \tag{56}$$

and for the optically thin case ($\tau_\nu \ll 1$), which holds for most directions away from the inner Galactic plane, this reduces to $T_\mathrm{B} \approx T_\mathrm{s}\tau_\nu$ and the integral over the line profile gives the column density directly:

$$\left[\frac{N_{\mathrm{H\,I}}}{\mathrm{cm^{-2}}}\right] = 1.823\times10^{18}\int\left[\frac{T_{\mathrm{B}}}{\mathrm{K}}\right]\left[\frac{\mathrm{d}v}{\mathrm{km\,s^{-1}}}\right]. \tag{57}$$

Equation (57) [9] is the single most useful result in 21 cm astronomy, and it is the reason spectra must be stored in linear power and calibrated in kelvin rather than decibels: the integral of a logarithm means nothing.

Table 28: Typical H I line parameters for the directions this telescope can reach

Direction	**Peak T_{B}** (K)	$\int T_{\mathrm{B}}\,\mathrm{d}v$ (K km s^{-1})	$N_{\mathrm{H\,I}}$ (cm^{-2})
Inner Galactic plane, $b = 0$	80–120	500–1600	1–3×10^{21}
Outer plane / anticentre	50–80	300–700	0.5–1.3×10^{21}
Intermediate latitude, $\lvert b\rvert \sim 30^\circ$	5–15	100–300	2–5×10^{20}
Coldest high-latitude sky	1–3	30–80	0.6–1.5×10^{20}

Two consequences of the 4.9° beam are worth stating plainly. First, this instrument never resolves individual clouds: it measures the beam-averaged column density over a patch of Galaxy some 5° across, which at the distance of the inner Galaxy is hundreds of parsecs. Second, the line profile it records is the superposition of everything along the line of sight, separated only in velocity — which is precisely what makes the next subsection possible.

17.3 Galactic rotation and the tangent-point method

Because the line is intrinsically narrow, its observed frequency encodes the radial velocity of the emitting gas, and Galactic rotation converts that velocity into position. For gas in circular rotation in the plane at galactocentric radius R, observed at Galactic longitude l,

$$v_{\mathrm{LSR}} = R_0\sin l\left[\frac{V(R)}{R} - \frac{V_0}{R_0}\right], \tag{58}$$

with $R_0 \approx 8.2\,\mathrm{kpc}$ the Sun's galactocentric distance and $V_0 \approx 235\,\mathrm{km\,s^{-1}}$ the local circular speed.

In the first quadrant ($0^\circ < l < 90^\circ$) the line of sight reaches a minimum galactocentric radius, the *tangent point*, at $R = R_0\sin l$, where the whole circular velocity lies along the line of sight. That gas produces the extreme positive velocity in the profile, and Equation (58) collapses to

$$V(R_0\sin l) = v_{\max}(l) + V_0\sin l. \tag{59}$$

Measuring the terminal velocity $v_{\max}$ as a function of longitude therefore traces the Galactic rotation curve directly, with no distance ladder and no assumption beyond circular rotation. It is the classic experiment for a telescope of this size, and everything in this document exists to make it possible: the pointing must be good to a fraction of the beam so that l means what it says, the clock must be right so that the alt-az conversion does not smear the pointing (Section 5), and the velocity axis must be corrected to the LSR (Section 9.5) or the derived rotation curve acquires an annual wobble of up to $30\,\mathrm{km\,s^{-1}}$.

A bandwidth of 2 MHz spans $\pm211\,\mathrm{km\,s^{-1}}$ and the default 2.4 MHz spans $\pm253\,\mathrm{km\,s^{-1}}$, comfortably covering the $\pm150\,\mathrm{km\,s^{-1}}$ that Galactic rotation produces. The default channel width of 586 Hz is $0.124\,\mathrm{km\,s^{-1}}$, some eighty times finer than the $\sim 10\,\mathrm{km\,s^{-1}}$ width of a typical line component — deliberately, because channels can be averaged after the fact but never un-averaged, and smoothing to $10\,\mathrm{km\,s^{-1}}$ buys a factor of nine in sensitivity.

17.4 What the telescope collects

The geometric area of a 3.0 m dish is $A_{\mathrm{g}} = \pi D^2/4 = 7.07\,\mathrm{m}^2$. The effective area follows from the beam through the antenna theorem, $A_{\mathrm{e}}\Omega_{\mathrm{A}} = \lambda^2$, with $\Omega_{\mathrm{A}} = \Omega_{\mathrm{MB}}/\eta_{\mathrm{MB}}$:

$$A_{\mathrm{e}} = \frac{\lambda^2 \eta_{\mathrm{MB}}}{1.133\,\theta_{\mathrm{FWHM}}^2} = \frac{0.04455 \times 0.7}{1.133 \times (0.0858)^2} = 3.74\ \mathrm{m}^2, \tag{60}$$

an aperture efficiency of $\eta_{\mathrm{A}} = A_{\mathrm{e}}/A_{\mathrm{g}} = 0.53$ — a reasonable value for a prime-focus dish with a simple feed, and the number the Sun measurement of Section 17.8 can check. The sensitivity in kelvin per jansky is then

$$\Gamma = \frac{A_{\mathrm{e}}}{2k} = \frac{3.74}{2 \times 1.381 \times 10^{-23}} = 1.35 \times 10^{-3}\ \mathrm{K\,Jy^{-1}}, \tag{61}$$

that is, **1.35 mK per jansky**. The reciprocal quantity, the system equivalent flux density, is

$$\mathrm{SEFD} = \frac{2k\,T_{\mathrm{sys}}}{A_{\mathrm{e}}} = \frac{T_{\mathrm{sys}}}{\Gamma} \approx 5.2 \times 10^4\ \mathrm{Jy} \quad \text{for } T_{\mathrm{sys}} = 70\ \mathrm{K}. \tag{62}$$

Fifty thousand janskys is a large number, and it correctly conveys that this is a small telescope: only the handful of brightest radio sources in the sky are detectable at all in continuum.

17.5 Expected antenna temperatures

For a source small compared with the beam, $T_{\mathrm{A}} = f\,\Gamma S$, where f is the beam-coupling factor. For a uniform disc of angular diameter θ_{s} observed with a Gaussian beam of width θ_{b},

$$f = \frac{1 - \exp\left(-\ln 2 \cdot x^2\right)}{\ln 2 \cdot x^2}, \qquad x = \frac{\theta_{\mathrm{s}}}{\theta_{\mathrm{b}}}, \tag{63}$$

which for the Sun or Moon ($\theta_{\mathrm{s}} = 0.53^\circ$, $x = 0.108$) gives $f = 0.996$ — a 0.4% correction, negligible beside the other uncertainties but free to apply. For a source that fills the beam, such as the H I line itself, the relation is instead $T_{\mathrm{A}} = \eta_{\mathrm{MB}} T_{\mathrm{B}}$.

Table 29: Expected antenna temperatures, using $\Gamma = 1.35\,\mathrm{mK\,Jy^{-1}}$ and $\eta_{\mathrm{MB}} = 0.7$. The Y column is the total-power ratio against blank sky for $T_{\mathrm{sys}} = 70\,\mathrm{K}$.

Source	Type	S or T_{B}	T_{A} (K)	Y
Sun (quiet)	disc, 0.53°	5.0×10^5 Jy	677	10.7
Sun (active)	disc	10–100× quiet	10^4–10^5	—
Cassiopeia A	compact	1660 Jy (2026)	2.25	1.032
Cygnus A	compact	1590 Jy	2.15	1.031
Moon	disc, 0.52°	890 Jy	1.20	1.017
Taurus A	compact	875 Jy	1.18	1.017
H I, inner plane	fills beam	80–120 K	55–85	1.8–2.2
H I, high latitude	fills beam	1–3 K	0.7–2.1	1.01–1.03
Galactic continuum, plane	fills beam	10–30 K	7–21	1.1–1.3

Cassiopeia A fades secularly at about 0.67% per year in L band, so its flux is computed from the Baars [10] epoch-1965 value with the Trotter [11] rate rather than tabulated; the simulator does exactly this, and the 2026 figure above comes from it.

Note the enormous dynamic range in that table. The Sun raises the total power by an order of magnitude; the next brightest object in the sky raises it by three per cent. That asymmetry matters more than it first appears, and the next subsection explains why it disqualifies almost everything in the table as a calibrator.

17.6 Why only the Sun and Moon can calibrate a 5° beam

A source list containing Cassiopeia A, Cygnus A and Taurus A invites the assumption that a small dish has four or five usable calibrators. It does not. At 4.9° the beam collects some 27 square degrees of sky, and all three of those sources lie within a few degrees of the Galactic plane, where the diffuse 1420 MHz continuum is far from negligible. **Pointing at them measures the Galactic background as much as the source.**

Table 30 makes this concrete, using the Stockert/Villa-Elisa survey that this repository already ships — with the strong sources removed from it, so the background column is genuinely diffuse emission — convolved to the telescope's own beam.

Table 30: Diffuse Galactic continuum in a 4.9° beam at the calibrator positions, from the compact 1420 MHz survey. The last column is the spread in background over eight off-positions one beamwidth away — the systematic an on/off measurement inherits from its choice of reference.

Source	l, b	T_A source	T_A background	Off-position spread
Cassiopeia A	111.7°, −2.1°	2.25 K	1.33 K	0.79 K
Cygnus A	76.2°, +5.8°	2.15 K	1.77 K	4.03 K
Taurus A	184.6°, −5.8°	1.18 K	0.70 K	0.16 K
Sun	(ecliptic)	677 K	< 5 K	< 5 K
Moon	(ecliptic)	1.20 K	< 5 K	*exactly zero*

The background is between 0.6 and 0.8 times the source signal in every case, and the off-position spread — the part that does not cancel — runs from 14% of the source for Taurus A to 190% of it for Cygnus A, which sits beside the Cygnus X star-forming complex. Cygnus A is simply unusable: whichever reference position is chosen, the answer changes by more than the thing being measured. Cassiopeia A carries a 35% systematic from the same cause, and Taurus A, at the anticentre, is the least bad of the three at 14% — still an order of magnitude worse than the 6–12% the solar method achieves.

The distinguishing property is not brightness, it is whether the background can be removed. A source fixed on the celestial sphere can never supply a reference measurement of the same piece of sky, because there is no epoch at which it is absent; every off-position is a different patch of Galaxy, and at this resolution those patches differ by a kelvin or more. The Sun and Moon escape by two different routes:

The Sun escapes by brightness
: At 677 K against a background of at most a few kelvin anywhere on the ecliptic, the contamination is below a per cent. No background subtraction is needed at all.

The Moon escapes by motion
: At 1.20 K the Moon is no brighter than Taurus A, and on its own that would make it equally useless. But it moves about 13° per day — a beamwidth every nine hours — so the *same celestial coordinates* can be observed with the Moon in the beam and again once it has left. The background then cancels identically rather than approximately, which is precisely what the fixed sources cannot offer. It is also a black body of known angular size at a physical temperature near 225 K, so its flux is predictable rather than measured.

Extragalactic source confusion, incidentally, is not the problem here: for a beam this size the classical confusion noise is of order 10^{-2} K, two orders below the diffuse Galactic variation. It is the Milky Way's own emission that spoils these calibrators, not the population of faint sources behind them.

The practical consequence is stated once and applies throughout the rest of this section: *the Sun is the calibrator, the Moon is the independent check, and the fixed radio sources are useful*

only as tests of gain stability — where a repeatable on/off difference matters and its absolute value does not.

17.7 Expected system temperature

Adding the budget of Section 9 to the sky contributions above:

Table 31: Expected system temperature in different circumstances

Condition	$T_{\mathbf{sys}}$ **(K)**	**Dominated by**
Cold sky, high latitude, near zenith	66–70	Receiver (43 K) and spillover (17 K)
Same, at 20° elevation	80–90	Ground spillover, atmosphere ($\sec z$)
Galactic plane, continuum only	75–90	Galactic synchrotron in the beam
Galactic plane, at H I line centre	130–160	The line itself
Beam on the quiet Sun	~ 750	The Sun
Beam on an active Sun	10^4–10^5	The Sun, and possibly saturation

The atmosphere contributes $T_{\rm atm}(1 - e^{-\tau_0 \sec z})$, and at 1.4 GHz the zenith opacity is only $\tau_0 \approx 0.01$, so with $T_{\rm atm} \approx 275$ K this is about 2.7 K at the zenith rising to 8 K at 20° elevation. It is a small term, which is one of the pleasant features of observing at this frequency, but it is not zero and it is elevation-dependent — so a system temperature quoted without an elevation is incomplete.

With $T_{\rm sys} = 70$ K the radiometer equation gives, per 586 Hz channel in 60 s, $\Delta T = 0.37$ K; smoothed to 10 km s^{-1} resolution the same minute gives 0.04 K. A high-latitude H I profile of 1–3 K peak is therefore a solid detection within a minute, and a well-measured profile within an hour.

17.8 Measuring the system temperature

A total-power measurement gives

$$P = G\,(T_{\rm sys} + T_{\rm A})\,, \tag{64}$$

with two unknowns, the gain G and $T_{\rm sys}$, and one equation. *One pointing can never determine the temperature scale.* Every method below is a way of obtaining a second, independent equation, and the differences between them are entirely about what has to be assumed.

17.8.1 Method A: ambient absorber

The most direct measurement of the receiver alone. Place microwave absorber at ambient temperature over the feed, then move it aside and look at cold sky:

$$Y = \frac{P_{\rm hot}}{P_{\rm cold}} = \frac{T_{\rm amb} + T_{\rm rx}}{T_{\rm cold} + T_{\rm rx}}, \qquad T_{\rm rx} = \frac{T_{\rm amb} - Y\,T_{\rm cold}}{Y - 1}. \tag{65}$$

With $T_{\rm amb} = 290$ K and $T_{\rm cold} \approx 6$ K (sky plus atmosphere), a receiver at 43 K gives $Y = 6.79$. The measurement is accurate and needs no astronomy at all, but note what it does *not* include: the absorber covers the feed, so spillover and ground pickup — the 17 K term, and the one most likely to be wrong — are excluded. It measures $T_{\rm rx}$, not $T_{\rm sys}$.

17.8.2 Method B: the Sun, of known flux density

This is the method this telescope is built for, since it already knows how to find and scan the Sun (Section 7). Compare the total power on the Sun with that on nearby blank sky:

$$Y_\odot = \frac{P_\odot}{P_{\rm sky}} = 1 + \frac{T_{\rm A,\odot}}{T_{\rm sys}}, \qquad T_{\rm sys} = \frac{f\,\Gamma\,S_\odot}{Y_\odot - 1}. \tag{66}$$

There is a subtlety here worth extracting, because it decides how much the result can be trusted. Substituting Equation (61) into (66) and comparing with Equation (62) gives

$$\boxed{\mathrm{SEFD} = \frac{f\,S_\odot}{Y_\odot - 1}} \tag{67}$$

— the effective area cancels entirely. **The Sun measures the SEFD directly, with no assumption about aperture efficiency**, and the SEFD is the quantity that actually sets what the telescope can detect. Converting that to a system temperature in kelvin, by contrast, requires $A_{\rm e}$, and it is $\eta_{\rm A}$ that carries the largest systematic error in the chain. The honest way to use this measurement is therefore to quote the SEFD as the primary result, and to treat $T_{\rm sys}$ and $\eta_{\rm A}$ as a pair — fix either one and the Sun gives you the other.

Four corrections and cautions apply.

The solar flux must be contemporaneous, not assumed
: The quiet Sun radiates about 5×10^5 Jy (50 solar flux units) at 21 cm, but an active Sun is ten to a hundred times brighter and varies on timescales of minutes. Fortunately the standard monitoring frequencies include 1415 MHz — the RSTN stations and DRAO both publish it daily [12] — which is within 0.4% of the observing frequency, so no spectral extrapolation is needed. Using a nominal quiet-Sun value on a day with an active region is the largest avoidable error in this method.

Point accurately, and do it after the pointing calibration
: A pointing offset Δ reduces the response by $\exp(-4\ln 2\,\Delta^2/\theta_{\rm b}^2)$, which is 2.8% at $\Delta = 0.5°$ and 11% at 1°. An uncalibrated mount, mispointing by the 2° of Section 7, would underread by 40%. Better still, use the peak of a Sun raster rather than a single pointing — the machinery already exists.

Take the reference close by in both angle and time
: The blank-sky reference should be a few degrees off the Sun and taken within a minute or two, so that gain drift, elevation-dependent spillover and atmospheric path all cancel rather than being corrected.

Correct for atmospheric attenuation
: The Sun is attenuated by $e^{-\tau_0 \sec z}$ on the way in, about 1% at the zenith and 3% at 20°. Small, but it acts in the same direction as the pointing error.

17.8.3 The two-point calibration: blank sky and the Sun as loads

The Y-factor of Equation (66) extracts one number, $T_{\rm sys}$, from the ratio of two measurements. The same two pointings, treated instead as two *known loads*, do considerably more: they establish the whole power-to-temperature scale, which is what turns the receiver's arbitrary linear units into kelvin.

Write the detected power as a linear response to the input temperature,

$$P = g\,T_{\rm in} + P_0, \tag{68}$$

with g the gain in units per kelvin and P_0 any offset that is present when no sky power is. Blank sky and the Sun give two points on that line:

$$P_{\mathrm{sky}} = g\,T_{\mathrm{sys}} + P_0, \tag{69}$$
$$P_{\odot} = g\,(T_{\mathrm{sys}} + T_{\mathrm{A},\odot}) + P_0, \tag{70}$$

and subtracting removes the offset entirely, leaving the scale factor as a pure difference measurement:

$$\boxed{k \equiv \frac{1}{g} = \frac{T_{\mathrm{A},\odot}}{P_{\odot} - P_{\mathrm{sky}}} = \frac{f\,\Gamma\,S_{\odot}}{P_{\odot} - P_{\mathrm{sky}}} \quad [\mathrm{K\ per\ unit}]} \tag{71}$$

The blank-sky level then reads

$$T_{\mathrm{sys}} = k\,(P_{\mathrm{sky}} - P_0)\,, \tag{72}$$

and *this* is where the offset matters. Equation (71) is immune to P_0; Equation (72) is not, and neither is the Y-factor, which tacitly sets $P_0 = 0$. If the detector carries a pedestal — a spectrometer noise floor, a DC spur at band centre, a quantisation offset — then the Y-factor overestimates the system temperature by exactly kP_0. The two measurements alone cannot reveal this; a third point can, and that is the argument for combining methods rather than choosing one.

Worked example. Suppose blank sky reads $P_{\mathrm{sky}} = 1.000$ in the receiver's arbitrary units and the Sun raster peak reads $P_{\odot} = 10.70$, on a day when the 1415 MHz solar flux is 50 sfu. Then $T_{\mathrm{A},\odot} = f\Gamma S_{\odot} = 0.996 \times 1.35 \times 10^{-3} \times 5.0 \times 10^{5} = 673\,\mathrm{K}$, and

$$k = \frac{673}{10.70 - 1.000} = 69.4\ \mathrm{K\ per\ unit}, \qquad T_{\mathrm{sys}} = 69.4 \times 1.000 = 69\ \mathrm{K}, \tag{73}$$

assuming $P_0 = 0$. That is the measurement the receiver's calibration box currently *assumes*: its single-point calibration (Section 9) asks the operator to type in a blank-sky temperature and derives $k = T_{\mathrm{cal}}/\bar{P}$ from it. The two-point procedure turns that entry from an input into an output — the same arithmetic, but with the 70 K established by observation rather than by a comment in `b210_h1_receiver.py`.

Applying the scale to spectra. For spectroscopy the two-point result is used in the standard position-switched form. With $P_{\mathrm{on}}(\nu)$ on source and $P_{\mathrm{off}}(\nu)$ on a nearby emission-free reference,

$$T_{\mathrm{A}}(\nu) = T_{\mathrm{sys}}\,\frac{P_{\mathrm{on}}(\nu) - P_{\mathrm{off}}(\nu)}{P_{\mathrm{off}}(\nu)}. \tag{74}$$

This form is worth more than it looks. Dividing by $P_{\mathrm{off}}(\nu)$ removes the instrumental bandpass — which is emphatically not flat across 2.4 MHz — at the same time as it sets the scale, and being a ratio it is again free of P_0 and of any slow change in g between the two. The two-point sky-and-Sun measurement supplies the single number T_{sys} that Equation (74) needs, and nothing else in the chain requires absolute power at all. This is the practical calibration pipeline for this telescope: measure T_{sys} against the Sun once per session, then observe H I entirely in on/off ratios.

Check the linearity before trusting either. Blank sky and the Sun differ by a factor of about ten in input power, whereas the H I line the calibration will be applied to sits within a factor of two of blank sky. A scale derived at the top of that range and applied at the bottom is only valid if the detector is linear across it. Two precautions follow, and both are cheap:

- **Fix the SDR gain and disable any automatic gain control.** The receiver already sets gain explicitly rather than leaving AGC enabled, but a gain chosen for H I may put the ADC near full scale on the Sun, and a compressed solar reading inflates k and deflates T_{sys}. Confirm the Sun measurement is not clipping before using it.
- **Take a third point in between.** It must be one whose input temperature is genuinely known, which rules out the fixed radio sources for the reason given in Section 17.6. Two qualify: the Moon ($\sim 1.2\,\mathrm{K}$, differenced against the same sky once it has moved on) and the Galactic plane continuum ($\sim$ 5–20 K, read from the survey and convolved to the beam as in Method C). Three points falling on a straight line through the origin validate both the linearity and the assumption $P_0 = 0$; three on a straight line with a non-zero intercept give the offset, after which Equation (72) can be applied properly.

Adding the ambient absorber of Method A as the hot point and blank sky as the cold one gives the same two-point structure with a load whose temperature is known to a fraction of a kelvin rather than to 5–10%. Its drawback is the one already noted — it sees no spillover — so the two calibrations are complementary: the absorber measures the electronics accurately, the Sun measures the whole telescope including everything the absorber bypasses, and the difference between them is an estimate of the spillover term itself.

17.8.4 Method C: cold sky and the survey slope

The third method uses only the sky, and exploits the fact that the 1420 MHz continuum brightness is already known everywhere from the Stockert/Villa-Elisa survey. Point at N directions of differing sky brightness and record the total power:

$$P_i = G\,(T_0 + T_{\mathrm{sky},i})\,, \tag{75}$$

where $T_{\mathrm{sky},i}$ is the survey brightness *convolved to this telescope's beam* — which is exactly what the simulator of Section 11 computes — and $T_0 = T_{\mathrm{rx}} + T_{\mathrm{spill}} + T_{\mathrm{atm}}$ collects everything that does not depend on where the dish points. A straight-line fit of P_i against $T_{\mathrm{sky},i}$ then yields both unknowns at once:

$$G = \frac{\mathrm{d}P}{\mathrm{d}T_{\mathrm{sky}}}, \qquad T_0 = \frac{P_{\mathrm{intercept}}}{G}, \qquad T_{\mathrm{sys}} = T_0 + T_{\mathrm{sky}}. \tag{76}$$

Its attraction is that it needs no absorber, no ephemeris and no external flux measurement, and that it isolates T_0 — the receiver-plus-spillover term that Method A cannot reach and Method B cannot separate. Its weakness is the lever arm: at 1420 MHz the sky ranges only from about 3.5 K at the coldest high-latitude minimum to perhaps 30 K beam-averaged in the inner plane, so a $\sim 25\,\mathrm{K}$ span is being used to calibrate a $\sim 70\,\mathrm{K}$ system. The gain must be stable to a few parts in a thousand across the whole set of pointings, which in practice means taking them within a few minutes and at similar elevations so that spillover does not vary underneath the fit.

17.8.5 Recommended procedure

For this telescope, the three methods are complementary rather than alternative, and the sensible order is:

1. Calibrate the pointing first (Section 7.9). Every method below assumes the beam is where it is thought to be.
2. Look up the day's 1415 MHz solar flux.

3. Run a Sun raster and take the fitted peak, immediately followed by a blank-sky reference 5–10° away at the same elevation. Repeat three or four times, alternating, over a few minutes.

4. Compute the SEFD from Equation (67). This is the primary, assumption-free result.

5. Take the same pair as a two-point calibration, Equation (71), to obtain the kelvin-per-unit scale k and hence T_{sys}. Enter that measured value in the receiver's calibration box instead of the assumed 70 K, and use it in Equation (74) for every spectrum of the session.

6. Add a third, intermediate point — the Moon, Cas A or the plane continuum — and confirm the three lie on a straight line through the origin. A non-zero intercept is a detector offset and must be subtracted before Equation (72); curvature means the Sun measurement is compressed and the SDR gain must come down.

7. Run a set of cold-sky pointings spanning as much of the 3.5–30 K range as the sky allows, and fit for G and T_0. This gives T_{sys} in kelvin independently of the Sun.

8. Combine: the two together over-determine the system, and their ratio yields A_{e} and hence the aperture efficiency — a genuine measurement of the antenna, rather than the assumed 0.53 used throughout this document.

9. Cross-check against the Moon, not against a fixed radio source. Observe the Moon's position, wait until it has moved a beamwidth — nine hours, or simply the following night — and repeat on the same celestial coordinates. The difference is the Moon alone, with the Galactic background subtracted identically rather than approximately, and at a predictable 1.20 K it is an independent test of the same scale the Sun gave. Section 17.6 explains why Cassiopeia A cannot serve here.

10. Use Cassiopeia A only as a *stability* test. Repeat the same on/off pair through an observing session: the absolute value is contaminated by the background and means little, but its repeatability at the 0.1 K level is good evidence that the gain is steady enough for the H I work.

Table 32: Uncertainty budget for the solar method

Term	**Typical**	**Reduced by**
Solar flux density	5–10%	Same-day 1415 MHz measurement
Pointing	1–3%	Raster peak rather than single pointing
Gain drift between on and off	1–2%	Rapid alternation
Beam coupling f	$< 1\%$	Equation (63)
Atmosphere	1–3%	Observing near the zenith
Detector offset P_0	0–5%	Third calibration point (Section 17.8.3)
Detector non-linearity	0–5%	SDR gain fixed, Sun not clipping
SEFD, combined	**6–12%**	
Aperture efficiency η_{A}	10–15%	Method C, or a second calibrator
T_{sys} in kelvin, combined	**12–20%**	

A system temperature known to 15% is entirely adequate for the science this telescope does: Equation (57) is linear in T_{B}, so a column density carries the same fractional error, and no 21 cm result from a 3 m dish rests on better than that.

18 The codebase

18.1 Repository and licence

The whole system — both firmwares, the receiver, the scheduler, the simulator, the enclosure and this document — lives in a single repository:

https://github.com/acrerd/21-cm-radio-telescope-v2

It is released under the MIT licence, copyright Acre Road Observatory. There is one long-lived branch, `main`; the observatory computer runs directly from a clone of it at `/home/astro/21-cm-radio-telescope-v2`, which is why several launcher scripts contain that path literally.

18.2 Working on GitHub

Outstanding work is recorded in GitHub issues and nowhere else. Two review documents that used to live in `docs/` were folded into issues and deleted; their content remains in the git history. Each issue carries the finding, `file:line` references, a suggested approach and the steps that verify it.

```
gh issue list
gh issue view 7 --comments
```

Listing 8: Reading the work queue

One label matters operationally. Issues marked `needs-hardware` cannot be closed on a development machine: they need a bench ESP32, the Due emulator of Section 13, or the mount itself, and elsewhere they are compile-check-only. Several issues also record conclusions that took considerable analysis and are easy to re-derive incorrectly — the reasoning is written out in each one, and re-deriving from first principles tends to reach the plausible-but-wrong version.

18.3 Layout

```
include/ src/ platformio.ini    Arduino Due drive firmware
esp32_controller_arduino/       WT32-ETH01 controller firmware
receiver_scheduler/             GNU Radio receiver, Flask scheduler, calibration
astro_simulator/                Desktop sky simulator
astro_simulator/web/            Browser port of the same
docs/                           Manuals and this article
enclosure/                      Printed case for the controller board
tools/                          Bench-test utilities
```

Listing 9: Top-level structure

The Due firmware sits at the repository root rather than in a subfolder because PlatformIO expects `src/` and `include/` beside its `platformio.ini`; the controller has its own `platformio.ini` and is therefore a separate project in the same tree.

18.4 Root and drive firmware

Table 33: Repository root and Arduino Due firmware

File	Purpose
`README.md`	Project overview, architecture sketch and quick start
`LICENSE`	MIT licence
`CLAUDE.md`	Conventions and hard-won constraints for anyone — human or AI assistant — editing the code. The prose equivalent of the "do not undo this" notes throughout this document
`.gitignore`	See Section 18.9
`.claude/settings.json`	Per-project assistant settings; currently empty
`platformio.ini`	Due build: environments `due` and `simulation`, and the DueFlashStorage dependency
`include/config.h`	Every pin, every default, the `Config` struct persisted to flash, and the simulation parameters
`src/main.cpp`	The entire drive firmware — state machine, homing, motion profiles, encoders, current sensing, faults, serial interface
`test_coordinates.ipynb`	Validates the controller's coordinate routines against `astropy`; the origin of the accuracy figures in Section 6

18.5 Controller firmware

Table 34: `esp32_controller_arduino/`

File	Purpose
`platformio.ini`	Environments `wt32-eth01` (default), `wt32-eth01-ota`, legacy `esp32s3`, and `native` for host tests
`partitions.csv`	Single-app flash layout
`partitions_wt32_ota.csv`	Dual-app layout, required for over-the-air updates
`data/index.html`	Standalone copy of the web UI. Not compiled in and no filesystem image is built — the served page is `src/index_html.h`
`.vscode/extensions.json`	Recommends the PlatformIO extension
`src/config.h`	Compile-time defaults: pins, ports, observer position, NTP servers, mount limits, horizon, stow
`src/main.cpp`	Application: the tracking loop, clock discipline, task setup
`src/state.h`	The single shared `SRTState` struct
`src/sync.{cpp,h}`	One recursive mutex covering everything touched by both `loopTask` and `async_tcp`, with the rules for using it
`src/settings.{cpp,h}`	Runtime settings and their NVS persistence
`src/coordinates.{cpp,h}`	Julian date, sidereal time, precession, alt/az, galactic, Sun, Moon, galactic-plane target
`src/pointing.{cpp,h}`	Refraction, the pointing model, and the true/drive frame transform — the boundary of Section 7
`src/srt_serial.{cpp,h}`	UART link to the Due: status parsing, target sending, the serial log
`src/web_server.{cpp,h}`	The HTTP API of Appendix C
`src/index_html.h`	The web interface, embedded as one string
`src/stellarium.{cpp,h}`	Stellarium telescope protocol on TCP 10001
`src/wifi_manager.{cpp,h}`	Access point and station management, credential storage
`test/shim/Arduino.h`	Host stand-in for the few Arduino facilities `pointing.cpp` uses
`test/shim/Preferences.h`	Host stand-in for NVS
`test/shim/shim.cpp`	Implementations behind those two headers
`test/test_coordinates/test_main.cpp`	Host tests of the coordinate transforms
`test/test_pointing/test_main.cpp`	Host tests of the pointing model and the frame round trip

18.6 Receiver, scheduler and calibration

Table 35: `receiver_scheduler/`

File	Purpose
`README.md`	Receiver and scheduler documentation
`requirements.txt`	Flask, PyEphem, pytest — what radioconda does not already supply
`b210_h1_receiver.py`	GNU Radio spectrometer: FFT, integration, live display, HDF5 logging, the calibration control of Section 9
`h1_web_scheduler.py`	The Flask scheduler: schedule, observation execution, telescope control, firmware updates, and the calibration endpoints
`sun_scan.py`	Sun raster, Gaussian fit, pointing-model fit, quality gates and the controller wire format
`test_scheduler.py`	Scheduler tests
`test_sun_scan.py`	Calibration tests, including the design matrix against numerically rotated coordinates
`pointing_data.json`	Accumulated Sun-scan records. The tracked set holds 23 scans from July and August 2026, all at `record_version` 1 — residuals against a model that is no longer knowable, so the fit skips them all (Section 7). They are kept for their measured beam widths, a median of 5.1°, which corroborate the 4.9° of Section 9.1
`read_h1_data.ipynb`	Reads HDF5 files, plots waterfalls and mean spectra
`start_scheduler.sh`	Launches the scheduler, reusing an already-serving instance
`start_srt_software.sh`	Desktop launcher for the whole software set
`start_platformio_monitor.sh`	Opens a serial monitor on the Due
`SRT Software.code-workspace`	VS Code workspace whose `folderOpen` task starts the scheduler

18.7 Sky simulator

Table 36: `astro_simulator/` — desktop version

File	Purpose
`README.md`	Usage, data provenance and the compression scheme
`requirements.txt`	NumPy, SciPy, matplotlib, astropy
`astro_simulator.py`	The interactive simulator of Section 11
`hi4pi_compact.npz.xz`	The HI4PI cube reduced from 33 GiB to ∼23 MB
`hi4pi_compress.py`	Regenerates that file from the full survey
`hi4pi_data.py`	Download helper for the full cube from CDS
`continuum_1420_compact.npz.xz`	Compact 1420 MHz continuum sky, ∼0.7 MB
`continuum_compress.py`	Regenerates it from the Stockert/Villa-Elisa survey
`nhi_grid_cache.npy`	Precomputed $N_{\mathrm{H\,I}}$ display map, so the window opens instantly
`run.sh`, `run.bat`	Launchers passing arguments through
`.gitignore`	Excludes the downloaded full-survey files

The browser port exists so the simulator can be used with no installation at all — a teaching consideration — and is deliberately restricted to the compact datasets, with no full-cube code paths.

Table 37: `astro_simulator/web/` — browser version

File	Purpose
`README.md`	How to serve and use it
`PLAN.md`	Design record and validation strategy
`index.html`	The page itself
`package.json`	Exists only so `node` runs the test harness; there is no build step
`js/main.js`	Bootstrap: fetch and decompress the bundles, wire the UI, read URL parameters in place of CLI flags
`js/skydata.js`	Sky and spectrum engine; a line-faithful port of the desktop `DishSimulator`
`js/coordinates.js`	Coordinate and time routines, ported from `coordinates.cpp`
`js/ephemeris.js`	Sun and Moon, plus the velocity-frame offsets
`js/map.js`	Mollweide all-sky canvas with overlays and click-to-point
`js/plot.js`	Spectrum and drift-scan panel
`js/ui.js`	Parameter row, targets menu, frame cycling, *Realise*
`make_web_data.py`	Converts the compact datasets into the web bundles
`data/hi4pi_web.bin.gz`	HI bundle
`data/continuum_web.bin.gz`	Continuum bundle
`data/meta.json`	Bundle geometry and scaling
`gen_golden.py`	Runs the desktop simulator at a fixed epoch to generate reference values
`test/golden.json`	Those reference values
`test/run_golden.mjs`	Checks the JavaScript engine reproduces every one of them

The golden-vector arrangement is the part worth noticing: the two simulators are independent implementations, and a fixed-epoch comparison is what stops them drifting apart.

18.8 Documentation, enclosure and tools

Table 38: `docs/`, `enclosure/` and `tools/`

File	Purpose
`docs/SRT_Technical_Article.tex`	This document
`docs/SRT_Technical_Article.pdf`	Its built form, tracked so it can be read without a LaTeX installation
`docs/update_version.sh`	Writes `version.tex`, the build stamp on the title page
`docs/SRT_DRIVE_MANUAL.md`	Operations manual for the drive firmware
`docs/SRT_DRIVE_MANUAL.html`	Its rendered form
`docs/ESP32_CONTROLLER.md`	Controller technical manual
`docs/WT32_ETH01_MIGRATION.md`	The ESP32-S3 to WT32-ETH01 migration, including the serial-flash recovery procedure
`docs/OBSERVATORY_HOST_SETUP.md`	Building the observatory host and its private link, Section 8
`enclosure/wt32_eth01_enclosure.scad`	Parametric OpenSCAD source for the controller case
`enclosure/*.stl`	Exported solid, base and lid
`tools/due_emulator.py`	The Due protocol emulator of Section 13

18.9 What is deliberately not tracked

Some absences are decisions rather than oversights, and each has cost someone time at least once.

Table 39: Untracked files that matter

Path	Why not tracked
`receiver_scheduler/scheduler_config.json`	Runtime configuration, and site-specific. It is the source of truth for the controller URL
`receiver_scheduler/h1_schedule.json`	The scheduler rewrites it on every edit and recreates it empty if absent. Ignored in both its real location and the repository root, where a stray copy appears if the scheduler is ever launched from there
`receiver_scheduler/pointing_model.json`	Refitted from `pointing_data.json` whenever it is needed
`receiver_scheduler/scheduler.log*`	Rotating log
`receiver_scheduler/data/`, `*.h5`	Observation data; large, and not source
`wifi_creds.json`	Credentials
`docs/version.tex`	Generated build stamp. Committing it would mean the document claims a commit hash that goes stale on the next commit
`.pio/`, `*.aux`, `*.idx`, ...	Build artefacts and LaTeX intermediates

19 Conclusion

The SRT drive controller is a layered system in which each layer holds exactly one kind of knowledge: the Due knows only pulses and motors, the controller knows the sky and the transform between the sky and the mount, and the scheduler knows what is to be observed and when. The two coordinate frames, and the single file where they meet, are the organising idea; most of the failures worth documenting here have come from mixing them.

Current capabilities:

- 0.5° encoder resolution with a fitted pointing model that removes the residual mount errors and a refraction correction applied as physics
- Absolute, model-free calibration data, so every fit stands on its own rather than being composed onto its predecessor
- J2000 coordinates throughout, compatible with modern catalogues and planetarium software
- Multiple control paths: web UI, REST API, Stellarium and the scheduler
- Automated scheduling, drift scans, satellite tracking and unattended data acquisition
- A private, reversible network link that keeps the controller's address and its clock under the observatory's own control
- Four hardware-free test paths — drive simulation, host unit tests, pytest and a Due emulator — covering everything but the motors
- A survey-backed sky simulator that predicts what the instrument should see, and can hand its pointing to the real dish

The instrument's principal remaining gap is calibration rather than control: the velocity axis is topocentric and the temperature scale rests on an assumed system temperature. Both are described in Sections 9 and 16, and both are addressed in analysis rather than at the telescope.

A Pin quick reference

```
Motor control:  8(PWM-Az), 9(DIR-Az), 10(PWM-Alt), 11(DIR-Alt)
Driver reset:   22(Az), 24(Alt)
Encoders:       12(Az), 13(Alt)
Fault flags:    5,4(Az FF1,FF2), 7,6(Alt FF1,FF2)
Current sense:  A0(Alt), A1(Az)
Serial1:        18(TX -> WT32 IO14), 19(RX <- WT32 IO4)
Calibrator:     26
```

Listing 10: Arduino Due pin summary

B Command quick reference

```
45 180              # Slew to Alt=45, Az=180
HOME                # Run the homing sequence
STOP                # Emergency stop
RESET               # Clear fault
CAL ON              # Calibrator noise source on
STATUS              # One status line now
CONFIG              # Show configuration
SET AZMAX 350       # Set a parameter
SET DEBOUNCE 50     # Azimuth encoder debounce (ms)
SAVE                # Save to flash
HELP                # Show all commands
```

Listing 11: Common Due serial commands

C HTTP API quick reference

C.1 Controller API

All endpoints are HTTP GET unless noted. Base URL `http://192.168.50.120`.

C.1.1 Status and information

Endpoint	Description
`/ping`	Minimal liveness check
`/network`	Ethernet and DNS diagnostics
`/status`	Drive `alt/az`, `true_alt/true_az`, RA/Dec, galactic l/b, target, currents, status, fault, `is_slewing`, calibrator, `pointing_loaded`, `clock_state`, `clock_age_s`, `lock_timeouts`, `malformed_status`
`/tracking`	Tracking state: enabled, RA/Dec, target name, axis mode, offsets, waiting flags
`/ephemeris`	Sun, Moon and galactic-plane (`plane`) positions
`/pointing`	The resident pointing model
`/serial/log`	Serial communication log
`/time/status`	`synced`, `source`, `sync_state`, `stale`, `last_sync_age_s`, `last_offset_ms`, `sync_count`, `utc`
`/settings`	All settings (Table 12)

C.1.2 Telescope control

Endpoint	Parameters	Description
/direct	alt, az	True alt/az slew; disables tracking
/goto	ra (h), dec	One-time slew to J2000 RA/Dec
/goto/galactic	l, b	One-time slew to galactic
/track/radec	ra (h), dec	Track J2000 RA/Dec
/track/galactic	l, b	Track galactic coordinates
/track/galactic-plane	—	Track the plane at the acquisition floor
/track/sun	—	Track the Sun
/track/moon	—	Track the Moon
/tracking/enable	enable (0/1)	Enable or disable tracking
/tracking/axis	mode = both, az or alt; alt, az	Track both axes or hold one fixed
/stop/movement	—	Stop motion, pause tracking commands 10 s
/stop/slewing	—	Stop the current slew
/stop/tracking	—	Stop tracking, leave the mount
/stop/all	—	Stop motion and clear the tracking target
/home	—	Run the Due physical homing sequence
/go-home	—	Slew to the stow position (drive frame)
/reset	—	Clear an active Due fault
/calibrator	on (0/1)	Noise source
/offset	alt, az	Sky-frame pointing offset
/offset/clear	—	Clear the offset

C.1.3 Pointing, time, network and settings

Endpoint	Parameters	Description
/pointing/apply	POST, JSON document	Replace and persist the model
/pointing/clear	—	Zero the model and erase NVS
/time/status	—	Clock health
/time/sync	—	Request an NTP poll now
/time/set	timestamp	Set the clock from the browser
/wifi/status	—	Ethernet and WiFi status, IPs, MACs
/wifi/scan	—	Scan for networks
/wifi/connect	ssid, password	Join a network
/wifi/forget	—	Clear saved credentials
/wifi/power	enable (0/1)	WiFi radio on/off
/eth/save	dhcp, ip, gateway, subnet, dns	Ethernet config (reboot to apply)
/settings/save	see Table 12	Save settings
/settings/reset	—	Reset settings to compiled defaults (the pointing model is *not* affected)

C.2 Scheduler API

Served on http://127.0.0.1:5000.

Endpoint	Method	Description
/api/schedule	GET/POST	Read or save the schedule; clashes rejected
/api/status	GET	Running state, current observation, seconds remaining
/api/start	POST	Start an observation immediately
/api/stop	POST	Stop the current observation
/api/stop_all	POST	Stop everything, including calibration
/api/receiver/status	GET	Whether a manual or observation-owned receiver runs
/api/receiver/start	POST	Manually boot the receiver for warm-up
/api/telescope	GET	Controller status, proxied

Endpoint	Method	Description
/api/config	GET/POST	Scheduler configuration (known keys only)
/api/fetch_tle	POST	Fetch a TLE from CelesTrak
/api/predict_pass	POST	Predict the next satellite pass
/api/drift_preview	GET	Drift-scan pointing and next transit
/api/log	GET	Last N lines of the scheduler log
/api/firmware/update	POST	Run the OTA firmware upload
/api/firmware/status	GET	Firmware upload progress
/api/sunscan/start	POST	Run one Sun scan
/api/sunscan/stop	POST	Cancel the running scan
/api/sunscan/status	GET	Scan progress and result
/api/sunscan/image	GET	Last scan image
/api/calday/start	POST	Start a calibration day
/api/calday/stop	POST	Stop the calibration day
/api/calday/status	GET	Phase, scans completed, next scan time
/api/calday/data	GET	Accumulated scan records
/api/calday/clear	POST	Discard accumulated scan records
/api/calday/fit	POST	Fit the pointing model and plot it
/api/calday/plot	GET	The calibration-day plot
/api/calday/apply	POST	Push the model to the controller
/api/calday/model	GET	The last fitted model

C.3 Examples

```
# Track RA/Dec (J2000), galactic coordinates, the plane, the Sun
curl "http://192.168.50.120/track/radec?ra=12.5&dec=-30"
curl "http://192.168.50.120/track/galactic?l=120&b=0"
curl "http://192.168.50.120/track/galactic-plane"
curl "http://192.168.50.120/track/sun"

# Direct alt/az slew (true frame), and park at stow (drive frame)
curl "http://192.168.50.120/direct?alt=45&az=180"
curl "http://192.168.50.120/go-home"

# Stop tracking; read status, ephemeris and clock
curl "http://192.168.50.120/tracking/enable?enable=0"
curl "http://192.168.50.120/status"
curl "http://192.168.50.120/ephemeris"
curl "http://192.168.50.120/time/status"

# Pointing model: read, apply, clear
curl "http://192.168.50.120/pointing"
curl -X POST "http://192.168.50.120/pointing/apply" \
  -H "Content-Type: application/json" \
  -d '{"version":1,"terms":{"IE":-0.72,"IA":1.98,"AN":0.31,"AE":-0.10}}'
curl "http://192.168.50.120/pointing/clear"
```

Listing 12: Controller API examples

```
# Schedule and status
curl "http://127.0.0.1:5000/api/schedule"
curl "http://127.0.0.1:5000/api/status"

# A calibration day, then fit and apply the model
curl -X POST "http://127.0.0.1:5000/api/calday/start" \
  -H "Content-Type: application/json" \
  -d '{"interval_minutes": 30, "sdr_type": "b210"}'
curl "http://127.0.0.1:5000/api/calday/status"
curl -X POST "http://127.0.0.1:5000/api/calday/fit"
curl -X POST "http://127.0.0.1:5000/api/calday/apply"

```

```
# Drift-scan preview
curl "http://127.0.0.1:5000/api/drift_preview?frame=radec&coord1=5.58&coord2
    =22.0&time=21:30"
```

Listing 13: Scheduler API examples

References

[1] J. Meeus, *Astronomical Algorithms*, 2nd ed., Willmann-Bell, 1998. Source of the solar and lunar ephemerides of Section 6.

[2] J. H. Lieske, T. Lederle, W. Fricke and B. Morando, "Expressions for the precession quantities based upon the IAU (1976) system of astronomical constants", *Astronomy & Astrophysics*, **58**, 1–16, 1977. The precession angles ζ_A, z_A, θ_A.

[3] S. Aoki, B. Guinot, G. H. Kaplan, H. Kinoshita, D. D. McCarthy and P. K. Seidelmann, "The new definition of Universal Time", *Astronomy & Astrophysics*, **105**, 359–361, 1982. The GMST expression used for sidereal time.

[4] G. G. Bennett, "The calculation of astronomical refraction in marine navigation", *Journal of Navigation*, **35**(2), 255–259, 1982. The refraction formula of Equation (28), here scaled by 1.15 for radio wavelengths.

[5] P. T. Wallace, "A rigorous algorithm for telescope pointing", *Proc. SPIE*, **4848**, 125–136, 2002. The alt-az pointing model terms IE, IA, AN, AE, CA, NPAE and TF.

[6] HI4PI Collaboration, "HI4PI: a full-sky H I survey based on EBHIS and GASS", *Astronomy & Astrophysics*, **594**, A116, 2016. The H I sky used by the simulator of Section 11.

[7] P. Reich and W. Reich, "A radio continuum survey of the northern sky at 1420 MHz", *Astronomy & Astrophysics Supplement*, **63**, 205–292, 1986; extended to the southern sky at Villa-Elisa by Reich, Testori and Reich, 2001. The 1420 MHz continuum sky used by the simulator.

[8] H. I. Ewen and E. M. Purcell, "Observation of a line in the galactic radio spectrum", *Nature*, **168**, 356, 1951. The detection of the 21 cm line.

[9] T. L. Wilson, K. Rohlfs and S. Hüttemeister, *Tools of Radio Astronomy*, 6th ed., Springer, 2013. Source of the antenna and radiometry relations of Sections 9 and 17, and of the column-density constant in Equation (57).

[10] J. W. M. Baars, R. Genzel, I. I. K. Pauliny-Toth and A. Witzel, "The absolute spectrum of Cas A", *Astronomy & Astrophysics*, **61**, 99–106, 1977. The flux scale for the calibrator sources of Table 29.

[11] A. S. Trotter et al., "The secular decrease of Cassiopeia A", *Monthly Notices of the Royal Astronomical Society*, **469**, 1299, 2017. The $0.670 \pm 0.019\%$ per year L-band fading rate applied to Cas A.

[12] US Air Force Radio Solar Telescope Network (RSTN) and the Dominion Radio Astrophysical Observatory solar flux monitoring programme. Daily solar flux densities at 1415 MHz — within 0.4% of the observing frequency, and the value required by Equation (67).

[13] F. R. Hoots and R. L. Roehrich, "Models for propagation of NORAD element sets", *Spacetrack Report No. 3*, 1980. The SGP4 propagator behind the scheduler's satellite tracking.

[14] Standards of Fundamental Astronomy (SOFA) Board, *SOFA Astrometry Tools*; and its ERFA fork, as used through `astropy` for the barycentric velocity correction of Section 9.5 and as the reference against which the on-board transforms are checked.

Index

21 cm line, *see* H I line

accuracy, pointing, 22
ADC clipping, 51
address plan, 28
AE (azimuth axis tilt, east), 23
ambient absorber, 49
AN (azimuth axis tilt, north), 23
antenna temperature, 47
antenna theorem, 47
aperture, 31
aperture efficiency, 47
architecture, 7
Arduino Due, 8
  pin assignments, 9, 59
atmosphere, 49
atmospheric attenuation, 50
atmospheric opacity, 49
automatic gain control, 51
azimuth coverage, 26
azimuth wrap-around, 20

backlash compensation, 15
bandpass correction, 51
bandwidth, 46
barycentric correction, 33
baud rate, 8
beam, 31
  averaging over the Galaxy, 46
  FWHM, 31
  solid angle, 31
beam coupling factor, 47
beam dilution, 31
blank-sky calibration, 32
branch, `main`, 54
brightness temperature, 45
building
  controller firmware, 40
  drive firmware, 39

CA (collimation error), 23
calibration, 23
  recommended procedure, 52
  two-point, 50
calibration day, 27
calibrator, 33
  recorded in metadata, 33
calibrators
  unusable at low resolution, 48
Cassiopeia A, 47
  secular fading, 47
CelesTrak, 36
channel averaging, 46
`CLAUDE.md`, 55
clock, 18
  error as pointing error, 18
codebase, 54
cold sky calibration, 52
column density, 45
communication interfaces, 8
condition number, 26
configuration
  drive firmware, 15
  persistence, 15
confusion, background, 48
continuum sky, 38
controller firmware, 16
coordinate frames, 6
  drive, 6
  true, 6
coordinate transformations, 21
CORS, 35
counterfeit hardware, 28
covariance, 26
cross-elevation, 25
cross-task locking, 16
  `sync.cpp`, 55
current sensing, 10
  filtering, 10
  hall-effect sensors, 10

data flow, 8
datum, absolute (T, D), 25
deadband, 17
design matrix, 25
detector offset, 50
DHCP, 29
direction inversion, 9
DNS, 19
dnsmasq, 29
drift scan, 36
drive firmware, 11
drive frame, *see* coordinate frames
`driveToTrue()`, 24
Due emulator, 41
DueFlashStorage, 15
dynamic range, 47

effective area, 47
Einstein A coefficient, 45
enclosure, printed, 11
encoder origin, 13
encoders, 10
  debounce, 10
  reed switch, 10
ESP32 controller, 10
ESPAsyncWebServer, 16
`espota`, 30
Ethernet, 18
Ettus B210, 34

fault flags, 9
fault injection, 41
faults, 16
  detection, 16
file lock, 34
firewall, 29
fixed-point iteration, 24
flash partitions, 55
forbidden transition, 45

FT232 programmer, 10
FWHM, 25

gain, receiver, 50
Galactic constants R_0, V_0, 46
Galactic continuum
  as a contaminant, 48
galactic coordinates, 22
Galactic longitude, 46
galactic plane, 20
  acquisition floor, 20
galactic rotation, 46
Gaussian fit, 25
geometric area, 47
`gh` CLI, 54
GitHub
  issue workflow, 54
  issues, 54
  labels, 54
  repository, 54
GitHub issues, 45
  needs-hardware, 42
`.gitignore`, 58
GMST, 21
GNU Radio, 34
golden vectors, 57
ground pickup, *see* spillover

H I line, 45
  hyperfine transition, 45
H-bridge drivers, 9
hardware, 8
HDF5, 34
  attributes, 34
  datasets, 34
HI4PI survey, 38
  compact cube, 38
homing, 13
  three-phase sequence, 13
horizon, observing, 20
horizontal coordinates, 22
hot/cold load, 49
hour angle, 22
HTTP API, 19, 59
hydrogen, neutral, *see* H I line

IA (azimuth index error), 23
ICRS, 21
IE (elevation index error), 23
installation, 39
integration time, 32
IP addresses, 28

J2000, 21
Julian date, 21

kelvin scale, 50
kinematic distance, 46
known limitations, 44

least squares, 25
  weighted, 25
level populations, 45
licence, 54
limit switches, 11
linearity, 50
local sidereal time, 21
low-noise amplifier, 32
LSR (Local Standard of Rest), 33
lwIP, 19

main-beam efficiency, 31
mDNS, 18
MIT licence, 54
Moon
  as calibrator, 52
  ephemeris, 22
motion profiles, 13
motors, 9
mount limits, 11

NAT, 29
native environment, 41
network card, 28
networking, 18
NetworkManager, 29
neutral hydrogen, 45
`nmcli`, 29
noise diode, 33
noise figure, 32
noise, radiometer, 32
NPAE (axis non-perpendicularity), 23
NTP, 18
NVS, 17
  pointing namespace, 24

observation parameters, 37
observatory host, 28
observer location, 40
observing, 42
on/off ratio, *see* position switching
operation, 42
optical depth, 45
OTA updates, 18, 30, 40
outstanding work, 44
overcurrent, 10

parameter significance, 26
pin assignments, 9, 59
PlatformIO, 39
  project layout, 54
pointing error
  in calibration, 50
pointing model, 23
  clamps, 24
  design, 23
  persistence, 24
  terms, 23
  wire format, 27
pointing offset, operator, 18
position switching, 51
precession, 21
private link, 28
process lock, 35
`PULSES_PER_DEGREE`, 10
PWM, 13
  inverted convention, 13
PyEphem, 36

pytest, 37

quality gates, 26

radial velocity, 46
radio velocity convention, 33
radioconda, 35, 40
radiometer equation, 32
radiometry, 31
ramps
  acceleration, 13
  deceleration, 13
raster scan, 25
Realise button, 39
receiver, 34
record version, 25
recovery, 30
reduced chi-squared, 26
refraction, 23
  Bennett's formula, 23
repository, 54
  layout, 54
rotation curve, 46
RSTN, 49
RTL-SDR, 34
runtime state, 58

safety features, 42
satellites, 36
scheduler, 35
  API, 60
  loopback binding, 8
  web interface, 35
SDR, 34
security, 8
SEFD, 47
sensitivity
  K per jansky, 47
serial commands, 14, 59
`SET` command, 15
settings, controller, 17
sfu, *see* solar flux unit
SGP4, 36
sidereal time, 21
simulation mode, 16
sky brightness, 52
sky simulator, 6, 38
  browser version, 56
  validation, 57
software limits, 11
solar flux density, 49
solar flux unit, 50
spillover, 32
spin temperature, 45
`SRTState`, 16
stall detection, 14
startup sequence, 42
state machine, 12
states
  DRIVING, 12
  FAULT, 12
  HOMING, 12
  IDLE, 12
  INIT, 12
status line, 15
  de-duplication, 15
  format, 15
  rate limiting, 15
Stellarium, 19
Stockert survey, 38
  as a calibrator, 52
stow position, 20
Sun
  active, 50
  as calibrator, 49
  ephemeris, 22
  quiet, 50
Sun scan, 25
system overview, 6
system temperature, 32
  expected values, 49
  measurement, 49
systemd, deliberately not used, 35

tangent-point method, 46
temperature calibration, 32
terminal velocity, 46
test shim, 41
testing, 40
TF (tube flexure), 23
time synchronisation, 18
TLE, 36
topocentric velocity, 33
total power measurement, 49
TP-Link TG-3468, 28
tracking, 20
troubleshooting, 43
  coordinates, 44
  motors, 43
  network, 44
  pointing, 43
  position, 43
  startup, 43
true frame, *see* coordinate frames
`trueToDrive()`, 24
two-point calibration, 50

UART link, 8
  line splicing, 42
`ufw`, 29
uncertainty budget, 52
unit tests, 41
untracked files, 58

velocity frames, 33
velocity resolution, 46
verification, network, 30

`waypipe`, 31
web interface, 42
WiFi access point, 18
WT32-ETH01, 10
  wiring to the Due, 10

Y-factor, 33, 49